A project report on

NEXUS: AN AI-DRIVEN PHYSICAL INTELLIGENCE PLATFORM FOR DIGITAL TWINS, RULE-BASED DECISIONS AND RELIABLE ACTUATION

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in

Computer Science and Engineering

by

RUDRAN M (23BCE1269)

BALAVIGNESH R (23BCE1802)

SUDHAN SRIRAM (23BRS1264)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

A project report on

NEXUS: AN AI-DRIVEN PHYSICAL INTELLIGENCE PLATFORM FOR DIGITAL TWINS, RULE-BASED DECISIONS AND RELIABLE ACTUATION

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in

Computer Science and Engineering

by

RUDRAN M (23BCE1269)

BALAVIGNESH R (23BCE1802)

SUDHAN SRIRAM (23BRS1264)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

DECLARATION

I hereby declare that the project report entitled “NEXUS: An AI-Driven Physical Intelligence Platform for Digital Twins, Rule-Based Decisions and Reliable Actuation”, submitted by BALAVIGNESH R (23BCE1802) for the award of the degree of Bachelor of Technology in Computer Science and Engineering at Vellore Institute of Technology, Chennai, is a record of bona fide work carried out by me as a member of the project team under the guidance of Dr. Malini Deepika. The report represents my individual submission and identifies the scope and evidence of the work performed.

I further declare that the work reported in this project has not been submitted, either in part or in full, for the award of any other degree or diploma at this or any other institution. The support of AI-based tools in the project is acknowledged in the Acknowledgement.

Place: Chennai
Date: 15-10-2026

Signature of the Candidate: _____ BALAVIGNESH R (23BCE1802)

CERTIFICATE

This is to certify that the project report entitled “NEXUS: An AI-Driven Physical Intelligence Platform for Digital Twins, Rule-Based Decisions and Reliable Actuation” is submitted individually by BALAVIGNESH R (23BCE1802), a student of Bachelor of Technology in Computer Science and Engineering at Vellore Institute of Technology, Chennai, in partial fulfillment of the degree requirements. The work was undertaken as part of the NEXUS project team under my guidance. The report is submitted for evaluation in accordance with university regulations.

Signature of the Guide: _____

Name: Dr. Malini Deepika

Date: _____

Signature of the Examiner 1: _____ Signature of the Examiner 2: _____

Name: _____ Name: _____

Date: _____ Date: _____

Approved by the Head of Department,

Computer Science and Engineering

Name: _____

Date: _____

(Seal of School of Computer Science and Engineering)

ABSTRACT

NEXUS is an AI-driven physical intelligence platform that connects physical devices to persisted, explainable and safely arbitrated decisions and delivers the resulting commands reliably. The software joining sensor readings to decisions, and decisions to physical commands, is often hard-wired to specific boards, difficult to test safely, poor at explaining why an action happened and unclear about what happened to a command after it was sent. NEXUS addresses these problems with an evidence-first design that was implemented and validated.

The platform is a modular monolith built with Spring Boot, PostgreSQL with TimescaleDB, Eclipse Mosquitto and a React Operations Center. Devices are described by data rather than code, through capability definitions, templates and per-device bindings with an onboarding workflow. A persistent Digital Twin keeps reported, desired and verification state with quality and freshness. The rule engine evaluates immutable rule versions against a frozen database snapshot, supports durable temporal operators, edge, cooldown and hysteresis policies, resolves conflicting actions by deterministic arbitration and persists every decision before any side effect. Accepted commands receive a durable delivery obligation written in the same transaction; a bounded, fenced dispatcher is the only publisher, records an append-only history, never reports an uncertain delivery as success, never resends it and holds later commands of the same device until the earlier message can no longer be applied by the firmware. Publication, acknowledgement, readback convergence and physical actuation are recorded as separate facts. A backend dry-run reuses the production decision logic without writing, a visual rule studio configures rules through the real backend, and an AI explanation path is advisory only: every claim is machine-checked against persisted evidence, with a deterministic fallback, and it has no route to activation or dispatch. A Python simulator and an ESP32 firmware speak the same MQTT contract, so the platform cannot tell them apart.

The system was validated with 1,479 backend tests on real PostgreSQL and Mosquitto containers, 533 frontend tests, a 72-scenario rule-engine acceptance ledger, a 152-scenario reliable-delivery ledger (62 scenarios covered, the rest planned for later slices) and an eleven-check validation gate, all passing, and with a simulated three-board deployment in which all eleven scenarios passed. The ESP32 firmware

implements the telemetry, presence and command contract, and bench bring-up confirmed the serial link, a button input and an LED output; the end-to-end physical measurements are recorded in the hardware evidence sheets only where they were captured. The prototype targets a trusted network; security hardening, retries with backoff, operational scaling and extended physical deployment are identified as future work.

Keywords: physical intelligence; digital twin; rule engine; MQTT; transactional outbox; IoT; cyber-physical systems; arbitration; dry-run simulation.

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, Dr. Malini Deepika, Department of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for her guidance, constructive feedback and encouragement during the development and evaluation of NEXUS. Her support helped us refine the project objectives, system design and verification approach.

I thank the School of Computer Science and Engineering at VIT Chennai for providing the academic environment and facilities needed for this project. I am also grateful to the faculty members and project reviewers whose observations helped strengthen the architecture, implementation and report.

I thank my teammates, Rudran M and Sudhan Sriram, for their collaboration throughout the project, and my family and friends for their continuous support. I also acknowledge the open-source tools, research publications and technical documentation referenced in this report.

AI-based tools supported coding, testing and drafting during the project, and all of their output was reviewed by the team. Responsibility for evaluating the work, verifying the stated evidence and submitting this individual report remains with me.

BALAVIGNESH R (23BCE1802)

Table of Contents

Chapter 1 Introduction 1
1.1 INTRODUCTION 1
1.2 BACKGROUND AND MOTIVATION 1
1.3 PROBLEM STATEMENT 2
1.4 OBJECTIVES 2
1.5 SCOPE OF THE PROJECT 3
1.6 EXPECTED OUTCOMES 4
1.7 APPROACH IN BRIEF 5
1.8 DIVISION OF WORK 5
1.9 ORGANISATION OF THE REPORT 5
Chapter 2 Literature Review 6
2.1 INTRODUCTION 6
2.2 CYBER-PHYSICAL SYSTEMS AND THE INTERNET OF THINGS 6
2.3 MESSAGING WITH MQTT 6
2.4 DIGITAL TWINS 7
2.5 RULE AND AUTOMATION ENGINES 7
2.6 RELIABLE MESSAGING AND CONSISTENCY 8
2.7 SAFE USE OF AI EXPLANATIONS 8
2.8 ARCHITECTURE STYLE 9
2.9 SUMMARY OF RELATED WORK 9
2.10 RESEARCH GAP 10
2.11 PATENT SEARCH 10
Chapter 3 Methodology and System Design 11
3.1 DEVELOPMENT METHODOLOGY 11
3.2 REQUIREMENTS 12
3.3 SYSTEM ARCHITECTURE 13
3.4 MODULE DESIGN 16
3.5 HARDWARE DESIGN 28
3.6 DATABASE DESIGN 34
3.7 API DESIGN 35
3.8 ARTIFICIAL-INTELLIGENCE SUBSYSTEM 35
3.9 CORE ALGORITHMS 40

3.10 OPERATIONAL FLOWS 41
3.11 CONFIGURATION AND SECURITY POSTURE 43
3.12 RELIABLE COMMAND DELIVERY (PHASE 4) 43
3.13 CORRECTNESS AND SAFETY PRINCIPLES 48
3.14 TEST AND VALIDATION DESIGN 48
3.15 FEASIBILITY, RISKS AND ETHICS 49
Chapter 4 Implementation and Results 51
4.1 IMPLEMENTATION ENVIRONMENT AND CODE SIZE 51
4.2 DEVELOPMENT CHRONOLOGY 51
4.3 KEY IMPLEMENTATION DETAILS 52
4.4 OPERATIONS CENTER SCREENS 53
4.5 TESTING AND VALIDATION 55
4.6 RESULTS 56
4.7 HARDWARE INTEGRATION RESULTS 59
4.8 COMPARISON AND ANALYSIS 61
4.9 EVALUATION OF THE AI EXPLANATION SUBSYSTEM 61
4.10 SELECTED TEST SCENARIOS 63
4.11 DISCUSSION 64
4.12 RELIABLE DELIVERY VALIDATION (PHASE 4) 64
4.13 LIMITATIONS AND THREATS TO VALIDITY 66
Chapter 5 Conclusion and Future Work 68
5.1 SUMMARY OF THE WORK 68
5.2 ACHIEVED CONTRIBUTIONS 68
5.3 SCOPE OF THE VALIDATED RESULTS 69
5.4 INTEGRATION OF SOFTWARE, HARDWARE AND AI 70
5.5 LESSONS LEARNED 70
5.6 FUTURE WORK 71
5.7 PROPOSED EXTENSION 71
5.8 PROPOSED ARCHITECTURAL EVOLUTION 73
APPENDICES 74
APPENDIX 1: ACCEPTANCE LEDGER OF PHASE 3 75
APPENDIX 2: PHASE 4 ACCEPTANCE LEDGER SUMMARY 77
APPENDIX 3: REPOSITORY STRUCTURE 78

APPENDIX 4: MQTT MESSAGE FORMATS 79
APPENDIX 5: SELECTED SOURCE LISTINGS 80
APPENDIX 6: HOW TO RUN AND VALIDATE THE SYSTEM 81
APPENDIX 7: HARDWARE EVIDENCE SHEETS 82
APPENDIX 8: INDIVIDUAL CONTRIBUTION AND PUBLICATION STATUS 83
A8.1 PUBLICATION AND PATENT STATUS 83
REFERENCES 84

List of Figures

Figure 3.1 Work plan by workstream (approximate weeks from 27 July 2026; workstreams overlap). 12

Figure 3.2 System context: devices, broker, backend, database and Operations Center. 13

Figure 3.3 The sense-to-verify loop and the durable record written at each stage. 14

Figure 3.4 Package dependency flow of the modular monolith. 15

Figure 3.5 Development and isolated validation environments. 16

Figure 3.6 Configuration-driven device model. 17

Figure 3.7 Digital Twin 2.0: reported, desired and verification facts with a derived effective view. 18

Figure 3.8 Command path and the five independent facts it produces. 19

Figure 3.9 Rule lifecycle with immutable versions. 21

Figure 3.10 The persisted rule decision path shared by typed and legacy triggers. 24

Figure 3.11 One decision engine shared by the production path and the dry-run path. 25

Figure 3.12 Transaction design of the dry-run before and after the connection-pool correction. 26

Figure 3.13 Evidence-grounded explanation pipeline with an optional AI reasoning branch. 27

Figure 3.14 Hardware architecture: components, their verification status and the shared MQTT contract. 30

Figure 3.15 Database tables grouped by responsibility. 35

Figure 3.16 Phase 4 command delivery: one acceptance transaction, a fenced dispatcher and three honest outcomes. 46

Figure 3.17 The uncertain-delivery hold relative to the firmware freshness limit. 47

Figure 4.1 Development timeline derived from the commit history 52

Figure 4.2 Digital Twins page (offline demo mode). 54

Figure 4.3 Rules page with the lifecycle and simulator entry points (offline demo mode). 54

Figure 4.4 Capability framework administration page (offline demo mode). 54

Figure 4.5 Backend automated test count at the recorded validation points. 55

Figure 4.6 Duration of each full-gate check. 56

Figure 4.7 System activity per minute during the Review III run. 57

Figure 4.8 Distribution of command dispatch processing time (n = 140). 58

List of Tables

Table 1.1 Scope of the reported work 3
Table 1.2 Division of work among team members 5
Table 2.1 Summary of related systems and their relevance to NEXUS 9
Table 3.1 The development cycle applied to every slice 11
Table 3.2 Functional requirements 12
Table 3.3 Non-functional requirements 13
Table 3.4 Technology stack and the reason for each choice 14
Table 3.5 MQTT topic contract 17
Table 3.6 Facts recorded for a command and what each does and does not prove 19
Table 3.7 Trigger families 21
Table 3.8 Temporal operators 22
Table 3.9 Candidate dispositions 23
Table 3.10 Dry-run request checks, all applied before any evaluation 26
Table 3.11 Main frontend areas 28
Table 3.12 Simulator device profiles 28
Table 3.13 Hardware inventory and role of each component 30
Table 3.14 Allocation of components to logical device profiles 31
Table 3.15 GPIO assignments and their basis 31
Table 3.16 Interface design for each module 32
Table 3.17 Behaviour of the committed firmware 32
Table 3.18 Failure handling across the hardware path 33
Table 3.19 Evidence levels for a hardware-driven action 34
Table 3.20 Design constraints of the AI subsystem 36
Table 3.21 What the mapper sends to a provider 36
Table 3.22 AI configuration properties 37
Table 3.23 Claim tiers and their shape rules 38
Table 3.24 Outcome codes of the explanation service 39
Listing 3.1 Arbitration, in outline 40
Table 3.25 Hysteresis example for a temperature condition (enter above 30, exit below 28) 41
Table 3.26 Tables added by Phase 4 (migrations V37 to V39) 44
Table 3.27 Delivery states and permitted transitions 45

Table 3.28 Publication outcomes and their recorded consequences (before retries exist)....46
Table 3.29 Invariants enforced by the design....48
Table 3.30 Evidence layers....49
Table 3.31 Principal risks and mitigations....49
Table 4.1 Implementation environment....51
Table 4.2 Repository size at the final checkpoint....51
Table 4.3 Milestones....52
Listing 4.1 Transaction boundary of the dry-run service (excerpt)....52
Listing 4.2 Bounded read in the dry-run controller....53
Table 4.4 Backend test count at checkpoints....55
Table 4.5 Full validation gate....56
Table 4.6 Review III scenarios (expected versus observed)....57
Table 4.7 Data produced during the Review III run....57
Table 4.8 Experiments on the dry-run engine....58
Table 4.9 Defects found and corrected during final review....58
Table 4.10 Hardware bench-test matrix and status....60
Table 4.11 Physical acceptance-test protocol....60
Table 4.12 Qualitative comparison....61
Table 4.13 Test classes of the AI explanation package....62
Table 4.14 Sample of Phase 3 ledger scenarios....63
Table 4.15 Phase 4 test groups....64
Table 4.16 Defects found in Phase 4 and their corrections....65
Table 5.1 Integration of the software, hardware and AI layers....70
Table 5.2 Future enhancements....71
Table 5.3 Proposed architectural changes....73
Table A1.1 Phase 3 acceptance scenarios....75
Table A2.1 Phase 4 ledger status by slice....77
Listing A3.1 Main directories of the repository....78
Table A4.1 Message examples....79
Listing A5.1 Rule decision function (RuleDecisionLogic.decide), shared by production and dry-run....80
Listing A5.2 ESP32 firmware: telemetry publication with QoS 1 (firmware/src/main.cpp)....80

Table A6.1 Operating commands.....81
Table A6.2 Commands used to validate the repository 81
Table A7.1 Evidence checklist.....82
Table A8.1 Division of work in the project team.....83

LIST OF ABBREVIATIONS

Abbreviation	Expansion
ACK	Acknowledgement
AI	Artificial Intelligence
API	Application Programming Interface
CPS	Cyber-Physical System
ESP32	Espressif 32-bit microcontroller family
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
JPA	Java Persistence API
JSON	JavaScript Object Notation
MQTT	Message Queuing Telemetry Transport
PIR	Passive Infrared (sensor)
QoS	Quality of Service
REST	Representational State Transfer
SQL	Structured Query Language
STOMP	Simple Text Oriented Messaging Protocol
TLS	Transport Layer Security
UI	User Interface
UUID	Universally Unique Identifier
YAML	YAML Ain't Markup Language

Chapter 1
Introduction

1.1 INTRODUCTION

Physical infrastructure such as buildings, laboratories and industrial plants is increasingly instrumented with low-cost networked sensors and actuators. A single microcontroller board can report temperature, humidity, gas concentration and motion every few seconds, and can switch a fan, a relay or a buzzer on request. The engineering difficulty is no longer reading a sensor; it is turning a continuous stream of measurements into trustworthy decisions and then into physical actions without causing harm. This report describes **NEXUS**, a software framework that was designed and built to address that difficulty.

NEXUS is organised around a simple processing loop: **sense, connect, understand, decide, act**, followed by **verification** of what the action actually achieved. A device publishes telemetry over MQTT; the backend validates it and stores it as typed history; a Digital Twin keeps the current, desired and derived state of every device together with an explicit quality judgement; a rule engine decides what should happen; guarded command dispatch sends the action back to the device; and the platform then records, as separate facts, whether the command was published, acknowledged and confirmed by a matching readback. Every stage persists its evidence in a relational database so that a later reader can answer why a decision was taken.

1.2 BACKGROUND AND MOTIVATION

Home-automation hubs, cloud IoT services and industrial platforms already ingest telemetry and run automations. Studying them, and building earlier versions of NEXUS, exposed recurring weaknesses that matter most when the controlled equipment is physical:

- **Device-specific code.** Many platforms hard-code a handful of sensor profiles. Adding a new kind of capability means editing and redeploying code.
- **Volatile state.** Rules, matches and twin state held in memory disappear when the process restarts, so history and in-flight work are lost.

- **Ambiguous command status.** A command that was published to a broker, acknowledged by firmware, and actually performed by an actuator are three different facts, but dashboards often present a single "success".
- **Unsafe rule evaluation.** Rules may read stale values, compare incompatible units, fire twice for one event, race with configuration changes or issue conflicting commands to one actuator.
- **No safe way to test a rule.** Operators can usually only test a rule by letting it act on real equipment.
- **Explanations that sound confident but are ungrounded.** A language model asked "why did this fire?" can invent a plausible but false reason.

NEXUS was started as a hands-on course project to explore how far these weaknesses can be removed by design rather than by convention. Its guiding principle is that the platform should never claim more than the evidence supports: a published command is not an acknowledged command, an acknowledgement is not convergence, and convergence is not proof that a physical actuator moved.

1.3 PROBLEM STATEMENT

The problem addressed by this project can be stated as follows.

Design and implement a simulation-first, configuration-driven framework that ingests heterogeneous device telemetry, maintains a persistent and quality-aware digital twin of each device, evaluates versioned rules deterministically against a coherent snapshot of that state, arbitrates the resulting actions safely, dispatches commands only under explicit authorisation, verifies their effect separately, and lets operators test and understand rules without touching production, while keeping every claim traceable to persisted evidence.

1.4 OBJECTIVES

1. **Generic device model (O1).** Replace hard-coded sensor profiles with capability definitions, device templates, per-device bindings and a guided MQTT onboarding workflow.

2. **Durable pipeline (O2).** Persist telemetry, events, rule matches, actions and pipeline traces in PostgreSQL/TimescaleDB with forward-only migrations, so that a backend restart loses nothing.
3. **Digital Twin 2.0 (O3).** Represent reported, desired and derived effective state, per-capability quality and freshness, a durable transition history, and readback-based verification of commands.
4. **Production-grade rule engine (O4).** Provide immutable rule versions with a lifecycle, typed triggers, a frozen evaluation context, temporal operators, cooldown, edge and hysteresis policies, mutual-exclusion arbitration and persisted action plans.
5. **Safe actuation (O5).** Make command acceptance depend on a persisted, authorised plan and keep acceptance, publication, acknowledgement, convergence and actuation as separate facts.
6. **Safe experimentation (O6).** Offer a backend dry-run that uses the same decision logic as production but has no ability to write, dispatch or publish.
7. **Grounded explanation (O7).** Explain rule matches from persisted evidence, with a deterministic fallback, and never let an AI component authorise anything.
8. **Evidence-based validation (O8).** Verify behaviour with automated tests on disposable infrastructure, a simulated three-ESP32 deployment, and a numbered acceptance ledger, and state honestly what is not proven.

1.5 SCOPE OF THE PROJECT

The scope is deliberately bounded so that every claim in this report can be backed by an executable check. Table 1.1 lists what is inside and outside the scope of the work reported here.

Table 1.1 Scope of the reported work

Area	Included	Not included
Devices	Python-simulated ESP32 profiles; ESP32 telemetry firmware, a PIR bench sketch and ESP32 bench bring-up (serial, button, LED); hardware bill of materials and electrical safety plan	Physical-actuation validation; a physical closed-loop ESP32 test; other protocol adapters

Area	Included	Not included
Platform	Modular monolith backend, TimescaleDB persistence, MQTT, realtime STOMP, REST API	Authentication, role-based authorisation, MQTT credentials and ACLs, rate limiting
Twin	Reported/desired/effective state, quality, history, command readback and convergence	Automatic desired-state clearing
Reliable delivery	Transactional command outbox, atomic acceptance, bounded fenced dispatcher, append-only command state history, uncertain-delivery barrier, strict feature flag with drain-only rollback (Phase 4: P4.0 to P4.2)	Retries with backoff, persisted ACK timeouts, telemetry deduplication, backpressure and delivery observability (planned slices P4.3 to P4.8)
Rules	Versions, lifecycle, typed triggers, temporal operators, arbitration, dry-run, explanations	Manual approval workflow, graph branching, cron/timezone scheduling, historical replay
Frontend	Operations Center, framework administration, rule studio, backend-driven simulator	Role-based views, tenant isolation, mobile layout
AI	AI-assisted decision support: evidence-grounded explanations of every rule decision (why a rule matched, why an action was selected or suppressed, what happened to the command), checked against persisted evidence, with a deterministic fallback ; AI-assisted rule creation (the AI drafts a rule that must pass the backend dry-run and operator review before activation); and guarded AI actuation (an AI-proposed action reaches a device only through an active rule, a persisted authorised plan and the existing command acceptance checks)	Learning from outcomes; the Jev risk and classification input (future scope)

1.6 EXPECTED OUTCOMES

- A working, tested platform in which a simulated device can be provisioned, can report telemetry, can be commanded and can have the effect of that command verified end to end.
- A rule engine whose decisions are reproducible, whose configuration changes cannot race with execution, and whose conflicting actions are resolved deterministically and recorded.
- A dry-run facility whose equivalence with production, and absence of side effects, are demonstrated by tests rather than asserted.
- An acceptance ledger of 72 numbered scenarios for the rule engine and a 152-scenario ledger for reliable command delivery, each scenario linked to the automated tests that cover it, and a clear list of limitations.
- A command-delivery layer in which an accepted command always has a durable delivery obligation, publication is bounded and fenced, and an

uncertain delivery is never reported as success, never resent and never overtaken by a later command.

1.7 APPROACH IN BRIEF

The work followed a **simulation-first** strategy. The backend does not distinguish a simulated device from physical hardware because both speak the same MQTT contract, so the whole software architecture could be exercised, repeated and measured before any hardware dependency was introduced. Each major capability was developed as a *contract*, a written statement of the required behaviour, followed by an implementation ticket, an acceptance ledger entry and automated tests. A feature counted as complete only when its exact test had been run and the result recorded. Independent forensic reviews of each milestone looked for defects, and several real defects found that way are reported in Chapter 4.

1.8 DIVISION OF WORK

The project was carried out by a team of at most three members under the guidance of the project guide. Table 1.2 records the division of work; each member also submits an individual contribution statement (Appendix 8).

Table 1.2 Division of work among team members

Member	Register number	Principal responsibilities
RUDRAN M	23BCE1269	Project team member; the individual contribution is stated in the member's own report.
BALAVIGNESH R	23BCE1802	Architecture and integration review, software validation and evidence, ESP32 bench verification (serial, button and LED), reliable-delivery validation review and the individual technical report.
SUDHAN SRIRAM	23BRS1264	Project team member; the individual contribution is stated in the member's own report.

1.9 ORGANISATION OF THE REPORT

Chapter 2 reviews the literature and existing platforms and identifies the gap addressed. Chapter 3 presents the methodology, requirements, architecture and the design of each module. Chapter 4 describes the implementation, the testing strategy and the results, including defects found during review and their correction. Chapter 5 concludes and lists future work. The appendices give supplementary material and the references close the report.

Chapter 2
Literature Review

2.1 INTRODUCTION

This chapter surveys the technical background on which NEXUS rests and the existing systems it is compared with. It covers cyber-physical systems and the Internet of Things, the MQTT messaging protocol, digital twins, rule and automation engines, reliable-messaging and consistency techniques, and the safe use of AI explanations. The review is based on published papers, standards and the public documentation of widely used platforms. Where a platform comparison relies on vendor documentation, the statement is limited to what that documentation describes.

2.2 CYBER-PHYSICAL SYSTEMS AND THE INTERNET OF THINGS

A cyber-physical system couples computation with physical processes through sensing and actuation (Lee, 2008). Lee argues that the timing, concurrency and reliability assumptions that are harmless in purely computational software become safety-relevant once software controls a physical process. Gubbi et al. (2013) describe the Internet of Things as a layered vision of sensing, networking, data storage and analytics, and Al-Fuqaha et al. (2015) survey its enabling technologies and protocols, identifying interoperability, scalability and security as open challenges. These works motivate two design choices in NEXUS: an explicit, device-independent capability model to address interoperability, and an explicit treatment of failure and uncertainty in the command path to address reliability.

2.3 MESSAGING WITH MQTT

MQTT is a lightweight publish/subscribe protocol designed for constrained devices and unreliable networks (Banks & Gupta, 2014). It defines three delivery guarantees: at most once (QoS 0), at least once (QoS 1) and exactly once (QoS 2). At-least-once delivery means a receiver can see the same message more than once, so handlers must be idempotent. Eclipse Mosquitto is a widely used open-source broker implementing the protocol (Light, 2017). NEXUS uses QoS 1 for telemetry and commands, a thin Eclipse Paho client rather than a heavyweight integration framework, and an external broker so that the broker can be scaled independently of

the application. The consequences of at-least-once delivery shape the acknowledgement, deduplication and recovery rules described in Chapter 3.

2.4 DIGITAL TWINS

The digital twin concept, introduced for product lifecycle management, describes a virtual representation of a physical object that is kept synchronised with it (Grieves & Vickers, 2017). Tao et al. (2019) review industrial digital-twin research and note that most published work concentrates on modelling and data fusion, with comparatively less attention to the control loop that returns decisions to the physical asset. Commercial and open-source platforms implement the idea in different ways:

- **AWS IoT Device Shadow** stores a JSON document per device with *reported* and *desired* sections and computes a *delta* between them, which lets a cloud application express intent while a device is offline (Amazon Web Services, n.d.).
- **Azure Digital Twins** models environments as a graph of twins described by the Digital Twins Definition Language and routes change events to other services (Microsoft, n.d.).
- **Eclipse Ditto** exposes devices as "things" with features and properties, a policy model and connectivity to brokers, as a framework for digital-twin back ends (Eclipse Foundation, n.d.).

These systems establish that separating reported from desired state is the right foundation. The documentation reviewed treats the gap between them as a delta to be resolved, and does not describe a notion of per-value quality or a qualified readback that distinguishes "the device said OK" from "the device state now matches the request". NEXUS extends the reported/desired model with exactly those two ideas.

2.5 RULE AND AUTOMATION ENGINES

Rule-based systems have a long history in artificial intelligence (Hayes-Roth, 1985). The Rete algorithm made large production systems efficient by sharing partial matches between rules (Forgy, 1982). IoT platforms use simpler, declarative automation models. Home Assistant describes automations as triggers, conditions and actions, and supports time-qualified triggers and execution traces (Home Assistant, n.d.). Node-RED composes flows of nodes in a visual editor (OpenJS Foundation,

n.d.). ThingsBoard processes messages through configurable rule chains (ThingsBoard, n.d.). These tools are valued for usability, and they are generally optimised for convenience in home and light-industrial settings.

Three aspects of rule execution recur as sources of subtle bugs and are central to NEXUS: (1) *level versus edge* semantics, that is, whether a rule that stays true acts repeatedly or once; (2) *debouncing* of noisy signals, for example with hysteresis thresholds; and (3) *conflict handling*, when two rules command the same actuator differently. Industrial control practice treats the third as a safety matter; NEXUS therefore resolves conflicts by deterministic, persisted arbitration with fail-closed tie handling.

2.6 RELIABLE MESSAGING AND CONSISTENCY

Kleppmann (2017) explains why distributed pipelines must tolerate duplicate delivery and crashes between steps, and describes idempotent consumers and the transactional outbox pattern as remedies (see also Richardson, 2018). NEXUS applies the outbox pattern to command delivery in Phase 4 (Chapter 3), and its earlier phases make each step idempotent by persisting a unique decision per trigger and by recovering unplanned work through a bounded scanner. For consistent reads, Berenson et al. (1995) formalised snapshot isolation, in which a transaction sees one committed state of the database; PostgreSQL implements this as REPEATABLE READ and extends it to serialisable snapshot isolation (Ports & Grittner, 2012). NEXUS uses REPEATABLE READ read-only transactions to give every rule evaluation, and every dry-run, a single coherent view of the world. Nygard (2018) catalogues stability patterns such as timeouts, bulkheads and circuit breakers; a connection-pool starvation defect found in the dry-run endpoint, described in Chapter 4, is a textbook example of the resource-exhaustion failure that book warns about.

2.7 SAFE USE OF AI EXPLANATIONS

Large language models can produce fluent explanations, but they can also generate unsupported statements, commonly called hallucinations (Ji et al., 2023). Retrieval-augmented generation reduces the problem by supplying retrieved evidence to the model (Lewis et al., 2020), but it does not by itself guarantee that every generated claim matches that evidence. Amodei et al. (2016) discuss accident risks in AI systems that act in the world and stress that systems should not be allowed to take

unsafe actions based on unreliable outputs. NEXUS follows two conclusions: the explanation component is **advisory only**, with no path to activation or dispatch, and its output is passed through a deterministic grounding check that compares every asserted fact with the persisted evidence it cites, falling back to a deterministic explanation when nothing survives.

2.8 ARCHITECTURE STYLE

Fowler (n.d.) recommends starting new systems as a well-structured monolith and extracting services only when a real need appears, because distributed systems add operational cost before the domain boundaries are understood. Evans (2003) describes bounded contexts and a ubiquitous language as ways to keep domain logic explicit, and Gamma et al. (1994) describe the observer and strategy patterns used by NEXUS for event dispatch and action executors. NEXUS is therefore built as a **modular monolith** with a one-way dependency rule between packages (Chapter 3), which keeps the option of extraction open without paying its cost up front.

2.9 SUMMARY OF RELATED WORK

Table 2.1 summarises the surveyed systems against the capabilities that matter for safe physical automation. The entries state what the reviewed documentation describes; absence of a feature in the table means it was not found in that material, not that the product cannot be extended to provide it.

Table 2.1 Summary of related systems and their relevance to NEXUS

System	Relevant strength	Not described in the reviewed material	Relevance to NEXUS
AWS IoT Device Shadow	Reported / desired / delta document per device	Per-value quality; readback-qualified convergence; rule arbitration	Confirms the reported/desired split
Azure Digital Twins	Twin graph, model language, event routing	Built-in rule decision and arbitration engine	Model-driven approach; routing of change events
Eclipse Ditto	Things/features, policies, broker connectivity	Versioned rules with a lifecycle	Framework-style device abstraction
Home Assistant	Triggers, conditions, actions, traces	Immutable versions; persisted arbitration; side-effect-free equivalence test	Automation vocabulary; usability benchmark
Node-RED	Visual flow editor, large node ecosystem	Persisted decision evidence and plans	Inspiration for the visual rule studio
ThingsBoard	Rule chains, dashboards, device profiles	Fail-closed conflict handling and dry-run equivalence proofs	Comparable end-to-end scope
NEXUS (this work)	All of the above in one auditable pipeline	Authentication; retries; physical actuation proof (future work)	Evidence-first design for physical safety

2.10 RESEARCH GAP

The review shows that device abstraction, digital twins and automation engines are individually well understood. The following gaps motivated the project:

1. **Quality-aware decisions.** Rules usually read the latest value without a standard way to refuse stale, offline or unit-incompatible evidence.
2. **Truthful command semantics.** Command acknowledgement, state convergence and physical effect are rarely separated and persisted as distinct facts.
3. **Auditable arbitration.** Conflicting actions are seldom resolved by a deterministic, persisted procedure whose outcome can be explained afterwards.
4. **Provable dry-run.** Testing a rule without side effects is common, but equivalence with real execution and the absence of side effects are rarely demonstrated by tests.
5. **Grounded explanation.** Explanations are seldom checked against stored evidence before being shown.

NEXUS addresses these gaps as an engineering contribution. It does not claim a new algorithm; its value lies in the careful composition of known techniques and in the evidence presented for their correctness.

2.11 PATENT SEARCH

No formal patent search was carried out for this project, and no claim of novelty in the patent sense is made. The work builds on openly documented standards and software, listed in the references.

Chapter 3
Methodology and System Design

3.1 DEVELOPMENT METHODOLOGY

The project was executed as a sequence of **phases**, each with a written goal, work items and exit criteria in a shared roadmap (Phase 0 clean baseline; Phase 1 generic device framework; Phase 2 Digital Twin; Phase 3 Rule Engine; Phases 4 to 9 planned). Within a phase, work proceeded in numbered slices (for example P2.1 to P2.4 and P3.0 to P3.6) that each followed the same cycle, shown in Table 3.1.

Table 3.1 The development cycle applied to every slice

Step	Activity	Artefact
1 Contract	Write the normative behaviour: states, invariants, failure handling, compatibility	Contract documents (Twin 2 contract, Rule Engine contract) and architecture decision records
2 Ticket	Order the work by dependency; define what each slice may and may not touch	Implementation ticket
3 Ledger	Enumerate testable scenarios before coding; status PLANNED until proven	Acceptance ledger (72 scenarios for Phase 3, 20+ for P2.4)
4 Implement	Backend, migration, frontend and simulator changes together	Source code, forward-only Flyway migration
5 Verify	Focused tests first, then the complete suite on disposable PostgreSQL and Mosquitto	Automated tests, validation reports
6 Review	Independent forensic review hunting for races, overstated claims and missing evidence	Findings, corrections, re-validation
7 Record	Mark a scenario COVERED only with exact test names and a recorded success	Updated ledger and ticket

Three principles governed every slice. First, **simulation first**: the backend cannot tell a simulated device from a real one, so behaviour is reproduced and measured in software before it is trusted on hardware. Second, **persist before you act**: a decision is written to the database before any side effect, so that crashes and replays are diagnosable. Third, **claim only what is proven**: statuses are recorded as separate facts, and the report distinguishes software evidence from physical evidence. The overall work plan is shown in Figure 3.1.

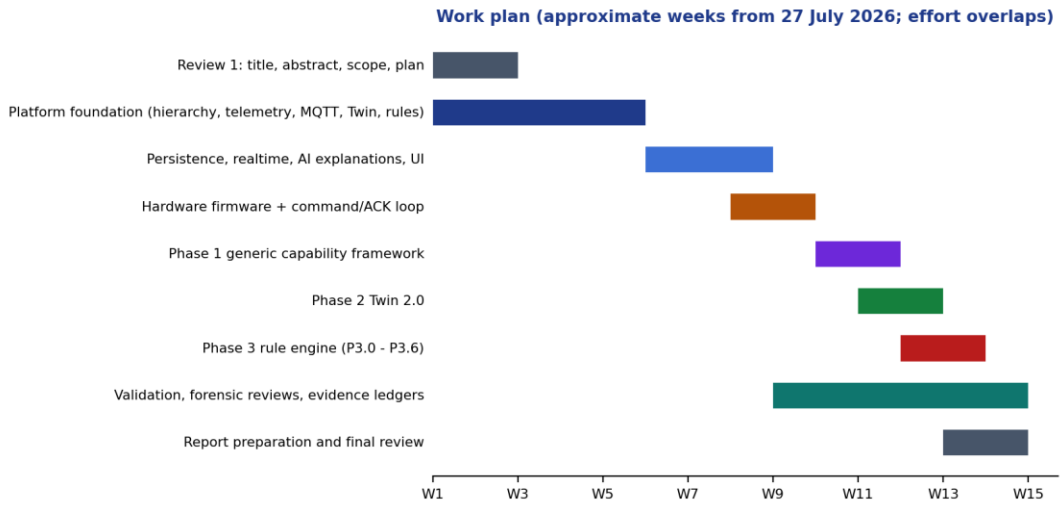


Figure 3.1 Work plan by workstream (approximate weeks from 27 July 2026; workstreams overlap).

3.2 REQUIREMENTS

The functional and non-functional requirements below were derived from the objectives in Chapter 1 and refined during the contract-writing step of each phase.

Table 3.2 Functional requirements

ID	Requirement	Objective
FR1	Manage a physical hierarchy of sites, buildings, spaces and devices through a REST API and UI	O1
FR2	Define capabilities, templates and per-device bindings without code changes; provision devices through an MQTT onboarding workflow	O1
FR3	Ingest telemetry over MQTT and HTTP, validate it against the device contract and store typed history	O1, O2
FR4	Maintain reported, desired and effective state per device, with quality and freshness, and persist every state transition	O3
FR5	Dispatch commands only after durable acceptance; record publication, acknowledgement and verification separately	O3, O5
FR6	Create rules as immutable versions with a lifecycle; evaluate them on typed triggers against a frozen snapshot	O4
FR7	Support temporal operators (duration, count, absence, offline, trend), cooldown, edge and hysteresis policies	O4
FR8	Arbitrate conflicting actions deterministically with mutual-exclusion groups and persist the plan	O4, O5
FR9	Provide a backend dry-run (supplied and current state) with no side effects and an unsupported-mode response for historical replay	O6
FR10	Explain a rule match from persisted evidence with a deterministic fallback; reject ungrounded claims	O7
FR11	Publish realtime updates to the UI and expose pipeline traces, analytics and health	O2
FR12	Provide a rule studio whose save goes through the real API and whose simulator uses the backend engine	O4, O6

Table 3.3 Non-functional requirements

ID	Requirement	How it is checked
NFR1	Durability: a backend restart must not lose devices, rules, matches, actions, traces or twin state	Restart/persistence smoke test; Review III restart scenario
NFR2	Correctness under concurrency: no duplicate decision or command for one trigger	Multi-transaction integration tests, unique-key constraints
NFR3	Safety: stale, offline or unit-incompatible evidence cannot satisfy a rule	Quality and unit-safety tests
NFR4	Isolation: validation never touches the development database	Testcontainers; startup scripts refused during validation
NFR5	Backward compatibility: legacy capability types and existing APIs keep working	Three-device fixtures, compatibility regression
NFR6	Evolvability: schema changes are forward-only and checksummed	39 Flyway migrations; migration tests on populated databases
NFR7	Observability: every pipeline step carries a correlation id and a trace	Pipeline traces, correlation tests
NFR8	Honesty: statements about physical effect are never inferred from software facts	Explanation and ledger rules
NFR9	Reproducibility: one command validates the repository (fast and full)	validate_nexus.py

3.3 SYSTEM ARCHITECTURE

3.3.1 OVERVIEW

Figure 3.2 shows the system context. Devices (physical ESP32 boards or the Python simulator) communicate with an Eclipse Mosquitto broker. The Spring Boot backend subscribes to device topics, persists data in PostgreSQL with the TimescaleDB extension, evaluates rules and publishes commands. A React single-page application, the Operations Center, calls the REST API and receives realtime updates over STOMP WebSockets.

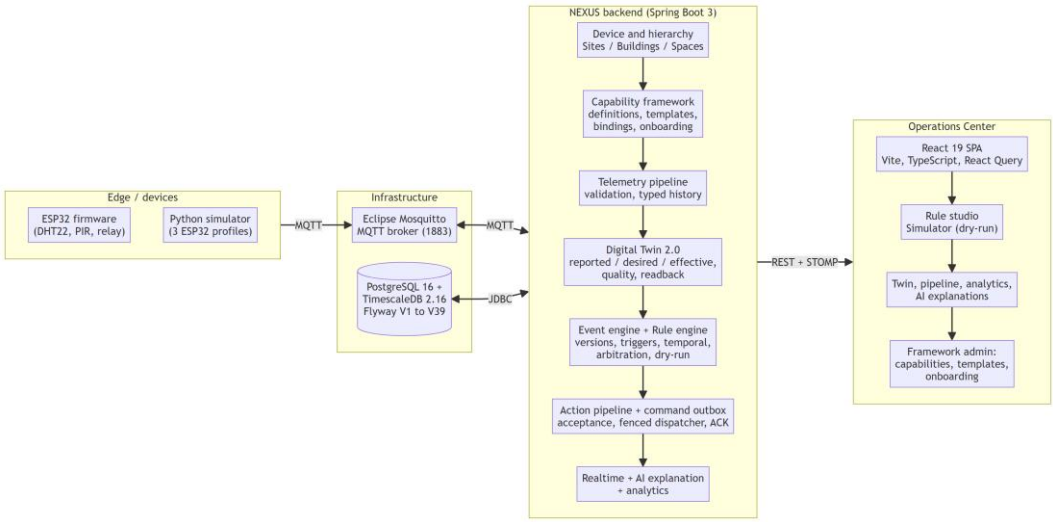


Figure 3.2 System context: devices, broker, backend, database and Operations Center.

Figure 3.3 traces the physical-intelligence loop through the persisted records that each stage produces. Evidence is written at every stage and joined by a correlation identifier, which is what makes a decision explainable after the fact.

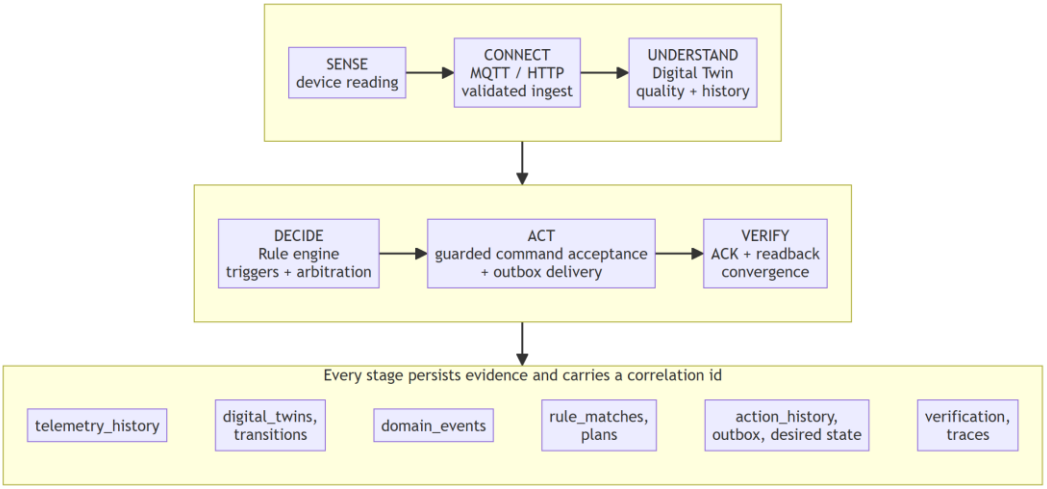


Figure 3.3 The sense-to-verify loop and the durable record written at each stage.

3.3.2 TECHNOLOGY STACK

Table 3.4 Technology stack and the reason for each choice

Layer	Technology	Rationale
Backend	Java 21, Spring Boot 3.3.4, Spring Data JPA, JDBC, Flyway	Mature transactional framework; strong typing for contracts; migration tooling
Messaging	Eclipse Mosquitto 2.x, Eclipse Paho client, Spring STOMP	Standard IoT protocol; broker scales independently; WebSocket push to the UI
Database	PostgreSQL 16 with TimescaleDB 2.16.1	ACID transactions, REPEATABLE READ snapshots; hypertables and retention for telemetry
Frontend	React 19, TypeScript, Vite, React Query, React Flow, Recharts, Tailwind CSS	Typed UI, server-state caching, graph editor for the rule studio
Simulator	Python 3.11, paho-mqtt, PyYAML	Concurrent virtual ESP32 devices speaking the same MQTT contract
Firmware	PlatformIO, Arduino framework for ESP32, 256dpi MQTT (QoS 1), ArduinoJson, DHT/BME280 libraries	Physical implementation of the telemetry contract
Validation	JUnit 5, Mockito, Testcontainers, Vitest, pytest, Python orchestration scripts	Real PostgreSQL and Mosquitto for integration evidence
Delivery	Docker Compose for infrastructure; start/stop/validate scripts	Reproducible local environment

3.3.3 MODULAR MONOLITH AND PACKAGE DEPENDENCIES

NEXUS is a single deployable Spring Boot application organised into packages that correspond to domain modules (Figure 3.4). Dependencies run in one direction, from domain modules towards shared and foundation packages; an upstream module such as the Digital Twin never depends on a downstream module such as the rule

engine. Communication from upstream to downstream is through immutable domain events published to a dispatcher. Decisions (rules) are separated from execution (actions): the rule engine never calls an executor directly, it persists a plan that the action pipeline then carries out. The repository contains 368 production Java files (about 21,300 lines), of which the rule package holds 140, the device package 80 and the twin package 37.

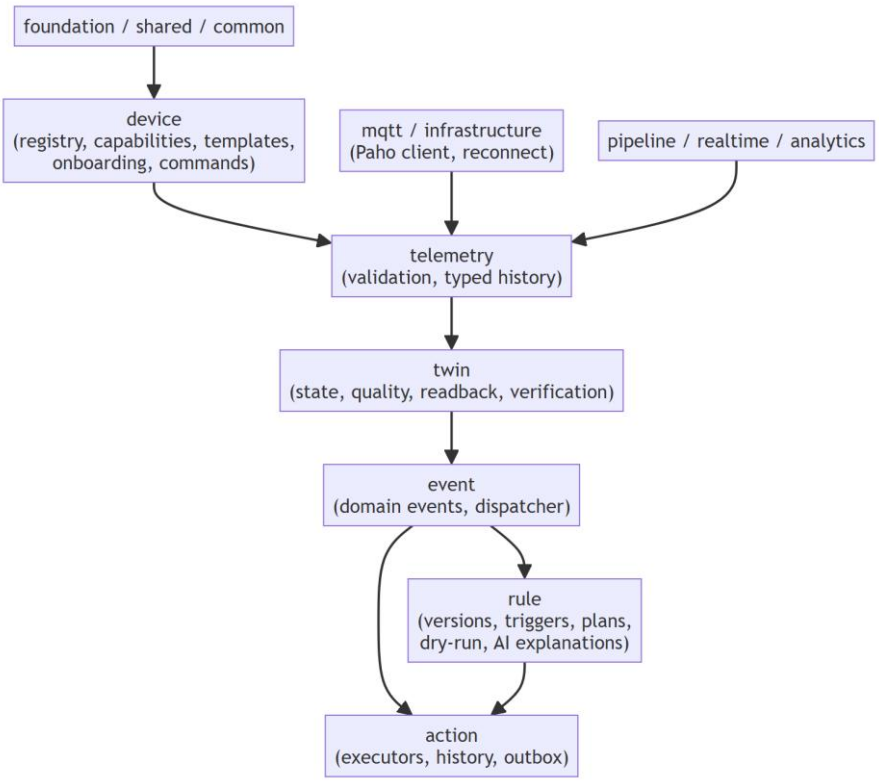


Figure 3.4 Package dependency flow of the modular monolith.

3.3.4 DEPLOYMENT AND TOOLING

The development environment (Figure 3.5) runs the backend on port 8080, the frontend on 5173, PostgreSQL/TimescaleDB on 5432 and Mosquitto on 1883. A Python orchestrator (`start_nexus.py`) refuses occupied ports, enforces a single live orchestrator, waits for bounded readiness and records ownership under an ignored runtime folder. A separate command (`validate_nexus.py`) runs the fast gate (eight checks) or the full gate (eleven checks), and refuses to run while the normal application is up so that validation never touches the development database. The MQTT broker is configured for trusted local use only.

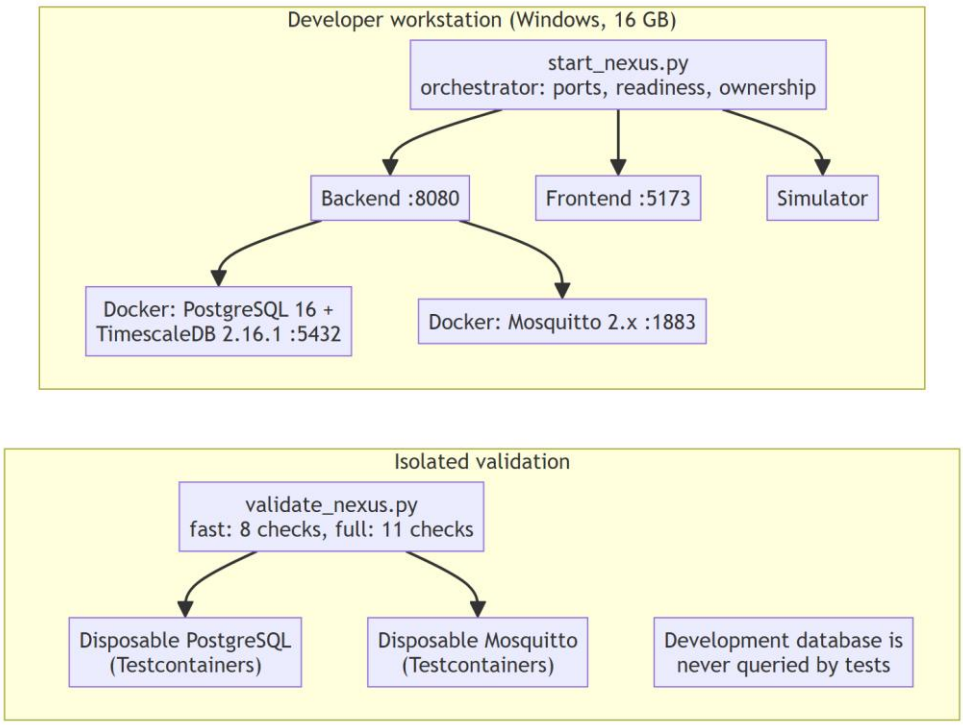


Figure 3.5 Development and isolated validation environments.

3.4 MODULE DESIGN

3.4.1 PHYSICAL HIERARCHY AND DEVICE REGISTRY

Sites contain buildings, buildings contain spaces and spaces contain devices. Each level has a REST resource, a repository and UI pages. Devices carry a serial number, a mode (simulated or hardware), a lifecycle status (REGISTERED, ACTIVE and others) and creation metadata. Deleting a parent that still has children is refused with a conflict response, so the hierarchy cannot be left inconsistent.

3.4.2 CAPABILITY FRAMEWORK

Early versions of NEXUS represented capabilities as a Java enumeration (temperature, humidity, pressure, gas, motion, button, LED, fan, buzzer, relay, servo). Phase 1 replaced this by data (Figure 3.6). A **capability definition** has a stable key, a direction (READ or WRITE), a value type (number, boolean, string, enumeration or object), a unit and allowed units, numeric limits, enumeration choices and the commands it supports. A **device template** composes definitions, and a provisioned device is an independent instance whose **bindings** may narrow, but never widen, the constraints of the definition and template. A single `RuntimeCapabilityResolver` is the migration boundary: it resolves an explicit binding first and falls back to the legacy adapter, so older devices keep working.

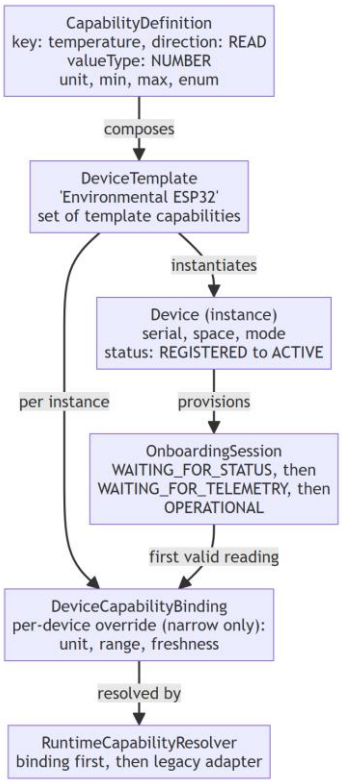


Figure 3.6 Configuration-driven device model.

Onboarding is a persistent session that moves through WAITING_FOR_STATUS, WAITING_FOR_TELEMETRY and OPERATIONAL. Provisioning creates the device in the REGISTERED state, its bindings and the session in one transaction and is idempotent for a repeated request key. A valid online status message advances the session; the first valid telemetry message is passed through the normal resolver and validator, stored, applied to the Twin, and only then does the same transaction activate the device. Invalid onboarding data is recorded as rejected without activating the device, and deadlines allow timeout and retry. The backend generates the MQTT contract for each device (Table 3.5).

Table 3.5 MQTT topic contract

Topic	Direction	Content	QoS
nexus/devices/{deviceId}/telemetry	device to backend	timestamp, sensor type or capability key, typed value, unit	1
nexus/devices/{deviceId}/status	device to backend	online/offline status used for connectivity	1
nexus/devices/{deviceId}/commands	backend to device	capability, command, value (for example a relay channel and state)	1
nexus/devices/{deviceId}/commands/ack	device to backend	command id and acknowledgement status	1

3.4.3 TELEMETRY PIPELINE

A reading arrives through `POST /api/v1/telemetry` or through the MQTT listener; both routes use one `TelemetryService`. The service checks that the device exists and is ACTIVE (or is the narrowly admitted onboarding device), resolves the capability, validates the typed value and unit against the definition and binding, stores the reading in the telemetry hypertable and updates the Twin. Malformed input is dropped early and, for onboarding devices, recorded as a rejection; a transient infrastructure failure is deliberately **not** classified as invalid data. The MQTT client uses a NEXUS-owned bounded reconnect controller instead of the library default, and a message that cannot be processed is not redelivered forever (a poison-message loop was found and fixed during development).

3.4.4 DIGITAL TWIN 2.0

The Twin keeps three kinds of fact and one derived view (Figure 3.7). **Reported state** is what the device last reported, with typed value, unit, device time and the exact backend receipt time. **Desired state** is what NEXUS has accepted as intent, stored durably before any command is published. **Verification** records whether a mapped readback confirms the desired state. The **effective state** is a derived structure that presents observation, intent and verification side by side; it is never a third writable truth.

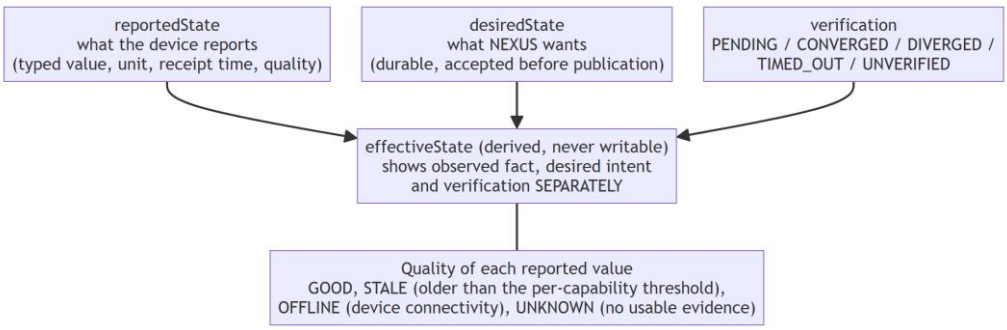


Figure 3.7 Digital Twin 2.0: reported, desired and verification facts with a derived effective view.

Every change is stored as a **transition** (previous and new value, source, correlation id, command id), which supports a cursor-based history API and a database-backed diff view. **Quality** is derived per capability from the receipt time and a freshness threshold (system default 120 seconds, overridable but only by narrowing): GOOD when the value is fresh and the device online, STALE when it is older than the threshold, OFFLINE when the device is offline and UNKNOWN when

there is no usable evidence such as a missing receipt time or unobserved connectivity. A rejected input is tracked as ingestion health and does not overwrite current state.

3.4.5 COMMAND, ACKNOWLEDGEMENT AND READBACK VERIFICATION

A command passes through the stages shown in Figure 3.8. Acceptance writes the desired state durably *before* publication; publication then sends the command to the broker; the device acknowledges; and, where an explicit mapping exists from a WRITE binding to a READ binding, a later readback with GOOD quality that matches the desired value establishes convergence. The status of each stage is stored in its own field (Table 3.6); no stage is inferred from another.

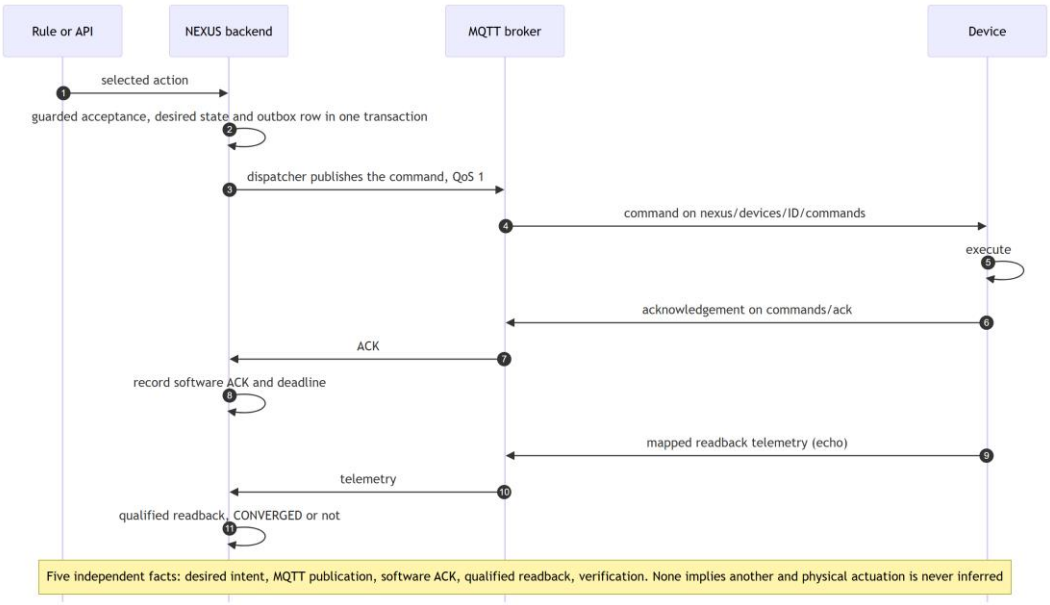


Figure 3.8 Command path and the five independent facts it produces.

Table 3.6 Facts recorded for a command and what each does and does not prove

Fact	Recorded by	What it proves	What it does not prove
Desired intent	Command acceptance	NEXUS accepted the request under guards	That anything was sent
Publication	Action history (SUCCESS)	The command was handed to the broker	That the device received it
Software ACK	ACK handler (receipt time and deadline)	The device software reported a status	That the actuator moved
Qualified readback	Readback mapping and verification state	A fresh GOOD observation matched the desired value (CONVERGED), or not (DIVERGED, TIMED_OUT)	Physical actuation by itself
Physical actuation	Not recorded by software	Nothing: remains UNKNOWN	Always requires independent evidence

Verification states are PENDING, CONVERGED, DIVERGED, TIMED_OUT and UNVERIFIED (for write-only capabilities with no mapping). Deadlines survive restarts. Desired state is never cleared automatically, and there is no retry or outbox in the current release, which is recorded as a limitation.

3.4.6 EVENT ENGINE

The Twin publishes immutable change events to a dispatcher. An event detector turns state transitions into domain events (for example twin created, twin updated, sensor value changed) which are persisted and delivered to subscribers such as the legacy rule adapter. Because subscribers implement a small listener interface, new consumers can be added without changing the Twin or telemetry code.

3.4.7 RULE ENGINE: VERSIONS, LIFECYCLE AND OWNERSHIP

A rule is a **logical identity** with immutable **versions**. Editing creates a new version with a higher revision; the previous content is never changed. The logical rule carries ownership data: the lifecycle state, references to the active and working versions, an optimistic row revision and an **execution epoch**. Pausing, resuming or activating a new version increments the epoch, so work scheduled under an earlier epoch is recognised as stale. Figure 3.9 shows the lifecycle. Execution is **persistence-gated**: only an ACTIVE version of the current owner with a passed reconciliation may run, and a legacy `PUT` of an enabled rule performs validation and activation atomically so that a failed activation leaves the previous version in force.

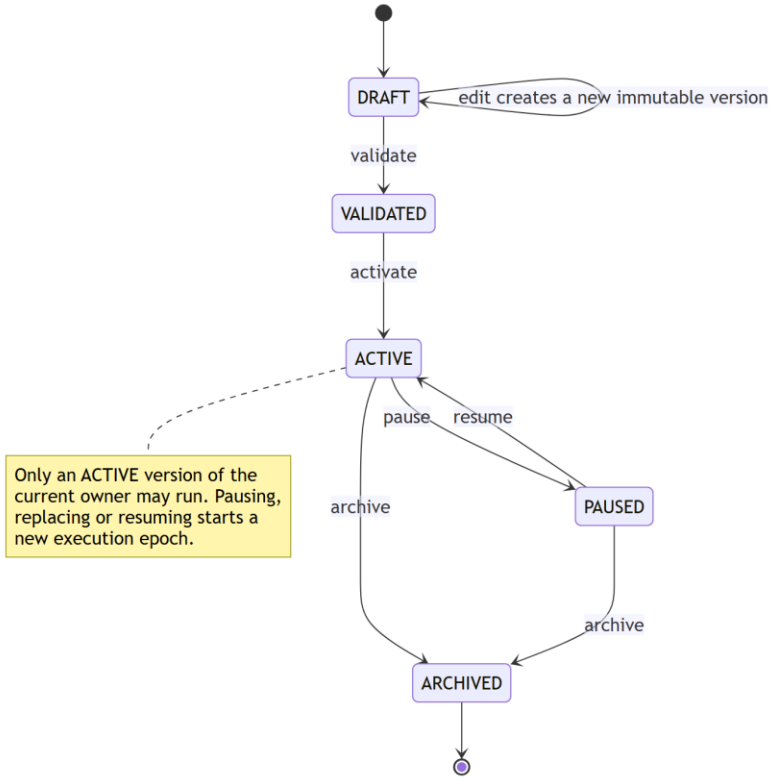


Figure 3.9 Rule lifecycle with immutable versions.

3.4.8 RULE ENGINE: TYPED TRIGGERS

Earlier versions evaluated every rule against every domain event. Phase 3 introduced thirteen explicit trigger families, and a version runs only for triggers it subscribes to (Table 3.7). A trigger is first recorded as a durable **fact** with a routing disposition; a recovery scanner re-routes facts whose processing failed, and expires those that are no longer eligible to act (for example a fact whose source has since been superseded).

Table 3.7 Trigger families

Family	Source
ACCEPTED_TELEMETRY, TWIN_STATE_CHANGED	A committed telemetry observation or Twin change (optional capability filter)
CONNECTIVITY_CHANGED	Device went ONLINE or OFFLINE
COMMAND_PUBLICATION_FAILED, COMMAND_ACK_REJECTED, COMMAND_ACK_TIMED_OUT	Outcome of a command on the action pipeline
VERIFICATION_CHANGED, VERIFICATION_TIMED_OUT	Readback verification result
MANUAL	A server-issued request addressed to the active version of one rule
SCHEDULE	A durable interval schedule (no cron or timezone support yet)
TEMPORAL_DUE	A temporal condition completed (duration elapsed, count reached, absence confirmed)

Family	Source
RULE_MATCHED	Another rule matched (explicit chains, depth at most 8, cycles rejected)
LEGACY_DOMAIN_EVENT	Compatibility adapter for rules created before Phase 3

3.4.9 RULE ENGINE: FROZEN EVALUATION CONTEXT AND EVALUATOR

For every trigger batch the engine acquires **one snapshot** inside a single REPEATABLE READ read-only transaction. The snapshot captures the eligible rule versions and owners, the Twin rows, connectivity with its generation, the installed capability contracts, quality inputs, temporal runtime rows and hysteresis latches. Evaluation then reads only the snapshot, never the live registry, so a decision cannot observe two different moments. The snapshot also decides whether the trigger source has already been superseded by a newer observation; if so nothing is evaluated for it.

The evaluator reads typed values. A capability condition is satisfied only when the value is present, its unit is compatible with the installed contract and its quality is GOOD. Unit safety is strict: equality of two missing units is accepted only for a capability whose contract is explicitly unitless, never as a way of ignoring an unestablished dimensional unit. Legacy payload conditions keep their original comparison semantics.

3.4.10 RULE ENGINE: TEMPORAL OPERATORS, REPEAT, COOLDOWN AND HYSTERESIS

Table 3.8 Temporal operators

Operator	Meaning	Bounds
FOR_DURATION	A predicate has been continuously true, with fresh coverage, for N seconds	Window at most 86,400 s
COUNT_WITHIN	At least N qualified samples (or false-to-true edges) inside a window	At most 200 members per state
ABSENCE_FOR	No qualified observation for N seconds	One timer per occurrence
OFFLINE_FOR	Device has been offline for N seconds	One timer per occurrence
TREND	Rising or falling by at least a minimum change over at least three distinct qualified samples, no interpolation	Bounded sample storage

Temporal state lives in the database (`rule_temporal_state` with evidence in `rule_temporal_evidence`) and deadlines in `rule_timers`, so it survives restarts and multiple instances. Late or out-of-order evidence is diagnosed and cannot rewind a window. Three policies prevent noisy or repeated firing. A **repeat policy** is either

EVERY_TRIGGER_WHILE_TRUE or EDGE_TRUE, where only a definite false observation re-arms the edge. A **cooldown**, global or per device scope, is stored in `rule_cooldown_state` and serialised by a row lock inside the match transaction so that two backend instances cannot both match. **Hysteresis** uses an explicit enter threshold and a strictly separated exit threshold on numeric conditions; its latch is stored per rule, condition and device and is updated only by definite enter or exit transitions, never by unknown or non-GOOD evidence.

3.4.11 RULE ENGINE: CANDIDATES, ARBITRATION AND PLANS

Every match yields one **candidate** per action. A command candidate has a *target key* (device, capability and, for relays, the channel) and an *equivalence key* describing the exact command. Arbitration is a pure, deterministic function over the batch (Table 3.9): identical commands on one target are coalesced; different commands on one target are decided by numeric priority, and an equal-priority conflict suppresses every contender (fail closed). **Mutual-exclusion groups** assign explicit integer ranks to rules; within a group at most one rule may act and an equal best rank suppresses all. Each candidate is judged against the same initial survivor set and by a total order, so the result is independent of group identifiers and input order; winner pointers always refer to a final selected candidate or are absent with an explicit reason.

Table 3.9 Candidate dispositions

Disposition	Meaning	Dispatches?
SELECTED	Chosen by arbitration (independent target, priority winner or coalesced primary)	Yes
COALESCED	Identical to a selected command; recorded as a contributor	No (primary acts)
SUPPRESSED_PRIORITY	Lost to a higher-priority command on the same target	No
SUPPRESSED_CONFLICT	Equal-priority conflict on the same target	No
SUPPRESSED_EXCLUSION	Lost its mutual-exclusion group to a lower rank number	No
SUPPRESSED_EXCLUSION_CONFLICT	Equal best rank inside a group	No

The decisions are written as an **action plan** keyed by the trigger fact (or, for legacy matches, the persisted event), with one plan per batch enforced by a unique constraint. Only SELECTED candidates reach **guarded acceptance**, which checks that the request equals the persisted candidate, that the rule is still owned and active under the same epoch, and that the source is still current. The result is recorded on the

candidate (ACCEPTED, EXECUTED, EXECUTION_FAILED, UNSUPPORTED, IGNORED or DENIED) and is derived from executor evidence: an ActionHistory row alone proves nothing, and a command is ACCEPTED only if its desired-state transition exists. A legacy match whose plan was never written is recovered by a bounded scanner (30 second grace, 120 second maximum age, five attempts) that rebuilds the batch from persisted evidence only. Unsupported policies, namely safety-override and approval configurations, are rejected at validation rather than silently ignored.

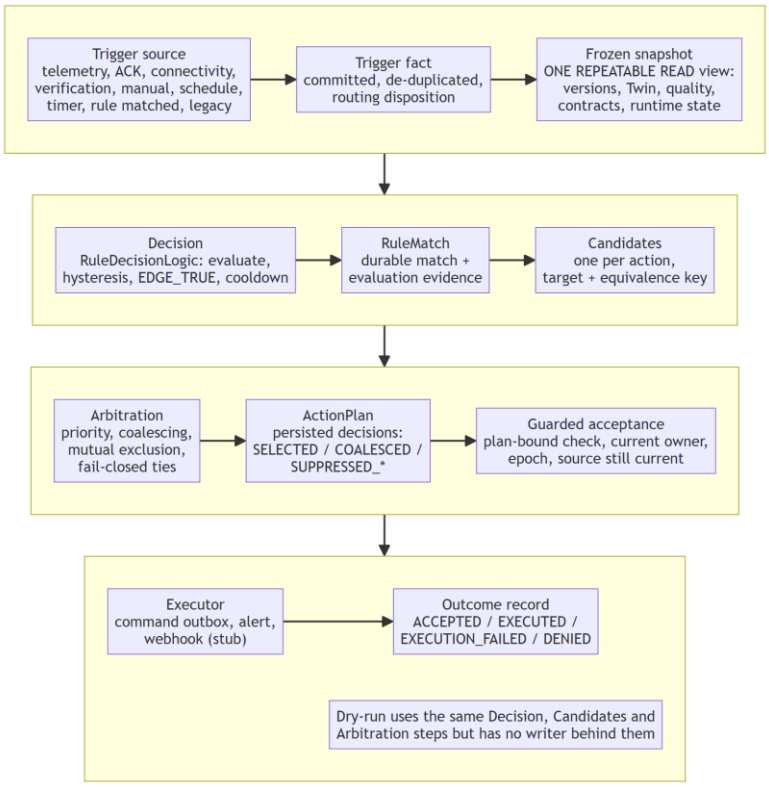


Figure 3.10 The persisted rule decision path shared by typed and legacy triggers.

3.4.12 BACKEND DRY-RUN

Testing a rule must not require letting it act. The dry-run endpoint `POST /api/v1/rules/dry-run` evaluates selected rule versions, stored or unsaved, and returns the evidence, the proposed candidates with their arbitration outcome and the hypothetical state changes production *would* make, without persisting anything. Two modes are supported. In **SUPPLIED_STATE** the caller provides typed Twin values, connectivity, an explicit simulation time and optional runtime seeds (cooldown, edge latch, hysteresis latch, temporal evidence); false, zero, an empty string, an explicit null and a missing value are all distinct. In **CURRENT_STATE** the backend uses its

own consistent snapshot and clock and copies the production runtime state, and refuses browser-supplied state. **HISTORICAL_REPLAY** is rejected with a typed error because the evidence needed for a faithful replay does not exist.

The design (Figure 3.11) is a *single implementation, two callers*. The evaluator, `RuleDecisionLogic`, `ActionCandidates` and `ActionArbiter` are shared verbatim; the production path wraps them in writers while the dry-run path wraps them in a pure engine whose only collaborators are read-only. Dry-run results use `sim-` identifiers that are not UUIDs and therefore can never be replayed as production command ids. Every candidate carries an `acceptanceAuthorization: NOT_EVALUATED` field, because SELECTED is an arbitration outcome and not a prediction that guarded acceptance would succeed.

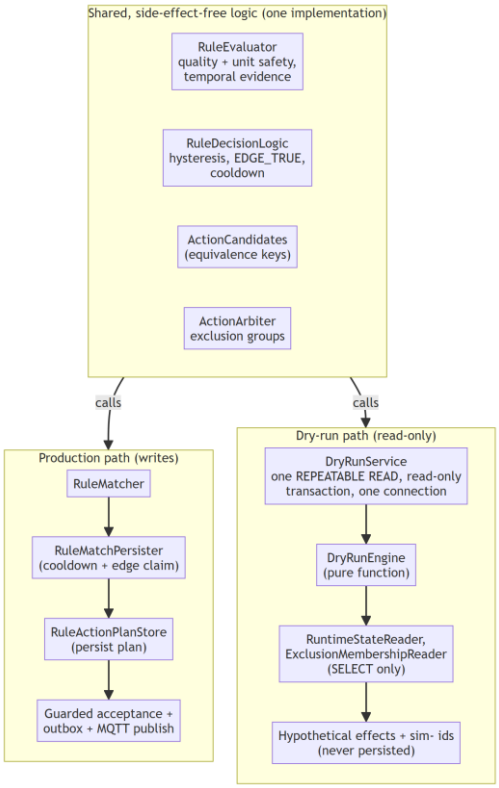


Figure 3.11 One decision engine shared by the production path and the dry-run path.

Safety against abuse is part of the design. The request is limited to one evaluation scope, at most 100 versions, at most 100 conditions per version and 1 MiB of input; the byte limit is enforced by a bounded read before any JSON binding (Table 3.10). The whole request runs as one REPEATABLE READ, read-only transaction on one pooled connection (Figure 3.12), so PostgreSQL itself refuses any write and the version lookup, snapshot, cooldown and exclusion reads agree with each other.

Table 3.10 Dry-run request checks, all applied before any evaluation

Check	Limit or rule	Response
Mode	SUPPLIED_STATE or CURRENT_STATE; HISTORICAL_REPLAY unsupported	400 / 422 with typed code
Versions	At most 100	413 TOO_MANY_VERSIONS
Conditions per version	At most 100	413 TOO_MANY_CONDITIONS
Scope	Exactly one device scope	400 MULTIPLE_SCOPES
Request size	At most 1 MiB (bounded read, no reliance on Content-Length)	413 INPUT_TOO_LARGE
Unknown fields	Rejected at every level, including inline definitions	400 INVALID_REQUEST
Supplied values	Typed: numbers must be JSON numbers, no coercion	400 INVALID_SUPPLIED_VALUE
Browser authority	CURRENT_STATE rejects supplied state and simulation time	400 CURRENT_STATE_REJECTS_SUPPLIED_INPUT

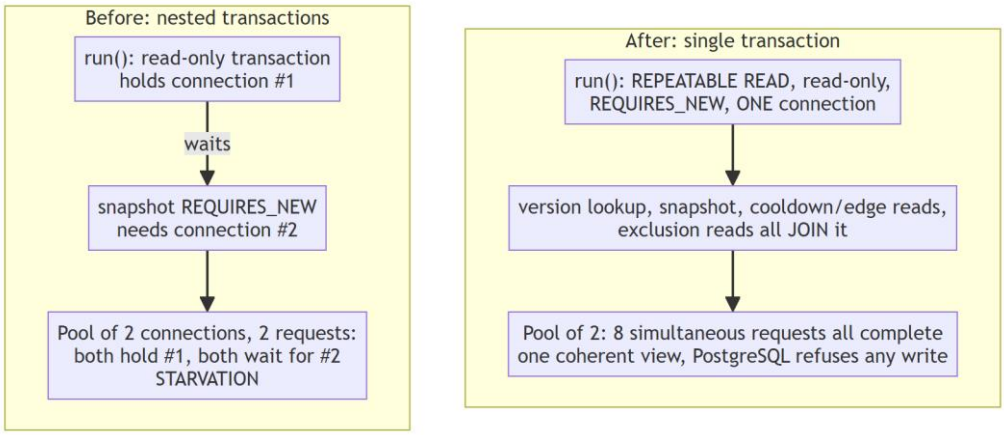


Figure 3.12 Transaction design of the dry-run before and after the connection-pool correction.

3.4.13 GROUNDED EXPLANATIONS

An explanation of a rule match is built from persisted rows only (Figure 3.13): the match, the rule revision and epoch, the evaluation evidence and its quality, the persisted candidates with their dispositions, winners and outcomes, the ActionHistory status, publication and acknowledgement, and the persisted verification state. The deterministic renderer produces claims of two main tiers: a **FACT** cites a path into the evidence and the exact value found there, and an **UNKNOWN** states what cannot be determined. When no arbitration record exists the explanation says so instead of inferring a selection from an ActionHistory row, and convergence is stated only from a persisted verification record. Physical actuation is always UNKNOWN. A platform-

owned note describes only executors that actually ran: a command is a real publication path while the alert and webhook executors are stubs.

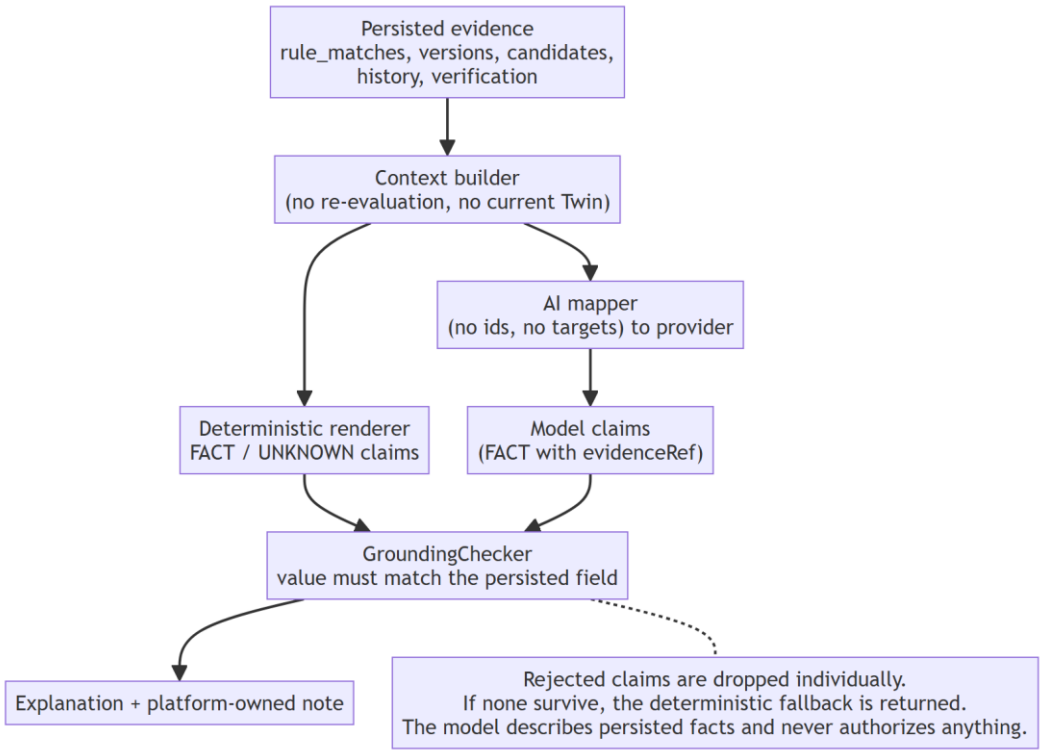


Figure 3.13 Evidence-grounded explanation pipeline with an optional AI reasoning branch.

When AI reasoning is enabled, a mapper removes identifiers and device targets before the context leaves NEXUS, and a deterministic grounding check compares every FACT claim with the value at the path it cites. Claims that do not match are rejected individually; if none survive, the deterministic explanation is returned. This check is structural: it proves the asserted value is present in the evidence, not that the surrounding sentence is a faithful paraphrase.

3.4.14 OPERATIONS CENTER (FRONTEND)

The frontend is a React single-page application with twenty-two routes (Table 3.11). Server state is cached with React Query and live updates arrive through a STOMP subscription that writes directly into the cache. The shipped default is the live backend; an opt-in demo mode driven by a scenario engine allows the interface to run without a backend. The rule studio is a graph editor built on React Flow that compiles to the flat definition the engine executes (all conditions, then all actions); branching graphs are refused rather than silently flattened. Its Save button calls the real create or update endpoint with the revision it loaded, and success is displayed only after the backend responds.

Table 3.11 Main frontend areas

Area	Routes	Purpose
Overview, Explorer	/, /explorer, site/building/space/device pages	Hierarchy navigation and device workspace
Digital Twins	/twins	Current state, quality, history, command and verification display
Telemetry, Events, Pipeline	/telemetry, /events, /pipeline	Time series, domain events, trace explorer
Rules and Actions	/rules, /actions, /framework/rules	Rule list, studio, exclusion groups, simulator, action history
Analytics and AI	/analytics, /ai	Aggregated metrics; evidence-grounded explanations
Framework administration	/framework/capabilities, /framework/templates, /framework/onboarding, /devices/:id/contract	Capability definitions, templates, onboarding sessions, generated MQTT contract
System	/settings, /system, /design, /about	Health, configuration and design system

3.4.15 SIMULATOR AND FIRMWARE

The Python simulator creates concurrent virtual ESP32 devices from a YAML description and publishes telemetry on the same topics as real hardware; it also handles commands, acknowledges them and, for mapped devices, emits correlated echo telemetry that lets the backend establish convergence. The default configuration defines three profiles (Table 3.12). The firmware directory contains a PlatformIO project for a real ESP32 with a DHT22 temperature sensor that publishes the identical contract with QoS 1, NTP-synchronised timestamps and reconnect handling, and a bench sketch for an HC-SR501 PIR sensor. The repository does not contain evidence of a physical closed-loop actuation test, so none is claimed.

Table 3.12 Simulator device profiles

Device	READ capabilities	WRITE capabilities
ESP32-SIM-001	temperature (CELSIUS), humidity (PERCENT), pressure (HPA), motion (BOOLEAN)	LED
ESP32-SIM-002	gas (PPM)	relay with channels 1 to 4, fan, buzzer
ESP32-SIM-003	button (BOOLEAN)	servo (DEGREES)

3.5 HARDWARE DESIGN

3.5.1 INTEGRATION OBJECTIVE AND END-TO-END PATH

The hardware subsystem connects real ESP32 nodes, sensors and low-voltage actuators to NEXUS through the **same MQTT contract that the simulator uses**, so

that a physical node is substitutable for a simulated one at the broker boundary and no rule, Twin or command logic changes. The complete path is shown in Figure 3.14 and consists of ten stages:

1. **Sensing:** an ESP32 reads a sensor (a DHT22 in the committed firmware; a BME280, MQ-2 and PIR in the integration specification).
2. **Transport:** the board publishes a JSON telemetry message over Wi-Fi to the Mosquitto broker with QoS 1.
3. **Ingestion:** the Spring Boot telemetry listener validates the device, capability, type and unit and rejects malformed input.
4. **Persistence:** the accepted reading is written to the telemetry hypertable in PostgreSQL.
5. **Digital Twin:** reported state, receipt time and quality are updated, and the transition is stored.
6. **Rule engine:** a typed trigger is evaluated against a frozen snapshot; matches are persisted.
7. **Plan and authorisation:** candidates are arbitrated and persisted as an ActionPlan; guarded acceptance authorises a selected command and stores the desired state.
8. **Command:** the backend publishes the command to the device command topic.
9. **Actuation:** the ESP32 validates the command against a closed allowlist and drives the output.
10. **Acknowledgement and readback:** the board publishes an ACK; a mapped readback observation, where a feedback signal exists, drives the verification state.

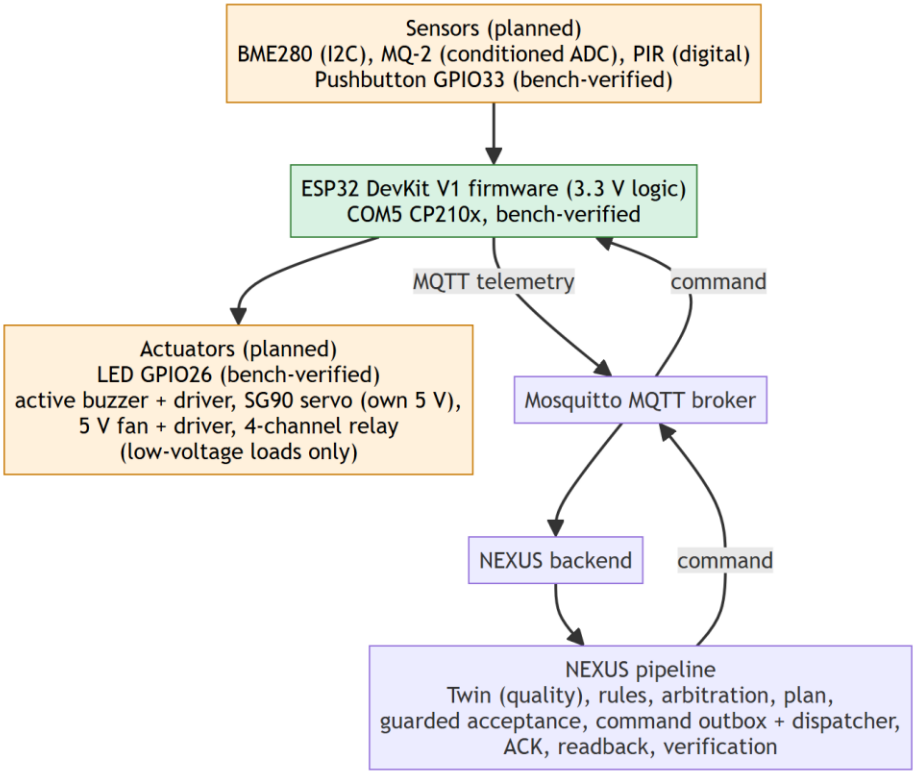


Figure 3.14 Hardware architecture: components, their verification status and the shared MQTT contract.

3.5.2 COMPONENT INVENTORY

Table 3.13 Hardware inventory and role of each component

Component	Quantity	Intended role	Principal electrical constraint
ESP32 DevKit V1 (ESP-WROOM-32)	3	Wi-Fi controller, one NEXUS node each	3.3 V logic on all GPIO and ADC pins
BME280 breakout	1	Temperature, humidity, pressure over I2C	Check breakout regulator and pull-ups before connecting
MQ-2 gas/smoke module	1	Qualitative gas/smoke response	5 V heater; analog output must be conditioned below 3.3 V; reading is not automatically ppm
PIR motion sensor	1	Motion events	Verify output level is 3.3 V compatible; warm-up required
4-channel relay module	1	Four independently addressed channels	Check coil supply, input polarity and 3.3 V compatibility; low-voltage isolated loads only
SG90 micro servo	1	Position command demonstration	Separate 5 V supply with transient headroom; common ground; never from the 3V3 pin
5 V DC fan	1	On/off actuator demonstration	Transistor or MOSFET driver with flyback protection; never from a GPIO
Active buzzer	1	Audible alert	Voltage and current to be confirmed; driver circuit required
LED module	1	GPIO and visual output check	Series resistor
Pushbuttons	5	Manual input	INPUT_PULLUP, active LOW
USB power bank	1	Candidate separate 5 V actuator supply	Current rating to be confirmed

3.5.3 MAPPING TO THE THREE NEXUS DEVICE PROFILES

The simulator defines three logical device profiles (Table 3.12). The physical components map onto the same three profiles, which keeps the rule definitions, bindings and test scenarios identical for simulated and real boards (Table 3.14). Which physical board owns which profile is a configuration decision made when the devices are provisioned, because the framework treats every board as an independently provisioned device with its own bindings.

Table 3.14 Allocation of components to logical device profiles

Profile	READ capabilities	WRITE capabilities	Physical components
Environment node (as ESP32-SIM-001)	temperature, humidity, pressure, motion	LED	BME280, PIR, LED
Safety node (as ESP32-SIM-002)	gas	relay channels 1 to 4, fan, buzzer	MQ-2, 4-channel relay, 5 V fan, buzzer with driver
Actuator node (as ESP32-SIM-003)	button	servo (degrees)	Pushbutton, SG90 servo

3.5.4 GPIO PLANNING

Table 3.15 records every pin assignment that exists, together with its source. Pins are grouped by how well they are established: pins used by committed firmware, pins used in recorded bench tests, and pins proposed in the integration plan. A proposed pin is a plan, not a verified wiring, and the final map must be fixed and checked before multi-component firmware is flashed. In particular GPIO26 is used for the LED bench test and must not also be assigned to the buzzer, and GPIO27 is used by the PIR bench sketch and therefore should not also carry the servo signal.

Table 3.15 GPIO assignments and their basis

Function	Pin	Basis
DHT22 temperature data	GPIO4	Committed firmware configuration template
Status/command LED	GPIO2	Committed firmware configuration template (LED command, safe default LOW)
PIR bench sketch input	GPIO27	Committed bench sketch in the repository
Pushbutton (INPUT_PULLUP, active LOW)	GPIO33	Recorded bench test
LED blink test	GPIO26	Recorded bench test
BME280 I2C (SDA, SCL)	GPIO21, GPIO22	Proposed (ESP32 default I2C pins)
MQ-2 analog input (conditioned)	GPIO34	Proposed (input-only ADC pin)
PIR input for integration	GPIO32	Proposed
Servo PWM signal, fan and buzzer drivers, relay inputs	to be fixed	Not allocated; final map required

3.5.5 POWER AND LOGIC-LEVEL INTERFACES

Four electrical rules apply to every circuit. ESP32 GPIO and ADC pins are a 3.3 V domain, so a 5 V signal is never connected directly. Actuator current never comes from a GPIO pin or the 3V3 pin; servo, fan, buzzer and relay coils are fed from a regulated supply through a suitable driver. Separately powered parts share a common ground, but two uncontrolled 5 V positive rails are never joined. Relay demonstrations are restricted to isolated low-voltage loads; switching of mains-powered equipment is outside the scope of this project. Table 3.16 gives the interface design for each module.

Table 3.16 Interface design for each module

Module	Supply	Signal interface	Protection and notes
BME280	3.3 V or as the breakout specifies	I2C, 3.3 V	Confirm pull-ups and regulator on the breakout
MQ-2	5 V heater, warm-up period	Analog output scaled below 3.3 V by a verified divider	Check ADC voltage with a multimeter; reading treated as a relative response, not ppm
PIR	Per module (typically 5 V)	Digital output, level verified 3.3 V compatible	Warm-up; motion is not guaranteed occupancy
4-channel relay	5 V coil supply	Input polarity and 3.3 V drive to be verified	Low-voltage isolated loads only; boot state must be off
SG90 servo	Separate regulated 5 V	3.3 V PWM	Common ground; range limits in firmware; ACK is not proof of movement
5 V fan	Regulated 5 V	Transistor or MOSFET gate drive	Flyback protection; no readback without a tachometer or similar sensor
Active buzzer	Per rating	Transistor drive	Direct GPIO drive was observed not to sound
LED	GPIO	GPIO via series resistor	Safe default LOW at boot

3.5.6 IMPLEMENTED ESP32 FIRMWARE

The repository contains a PlatformIO firmware project for the ESP32 (Arduino framework, libraries pinned to exact versions: 256dpi MQTT 2.5.2, ArduinoJson 7.2.1, Adafruit DHT 1.4.6 and Adafruit BME280 2.2.4). Its implemented behaviour, and what it deliberately does not do, is summarised in Table 3.17. All Wi-Fi, broker and identity values live in an ignored configuration file, so no secret is committed.

Table 3.17 Behaviour of the committed firmware

Function	Implementation
Connectivity	Wi-Fi connect with a 20 s timeout and retry; MQTT connect with a 10 s timeout; client identifier derived from the device UUID so several boards never collide
Time	NTP synchronisation; a plausibility floor on the epoch; telemetry and commands are not processed while the clock is unsynchronised, so no untrustworthy timestamp is published

Function	Implementation
Telemetry	DHT22 temperature every 5 s in the NEXUS wire contract at QoS 1; a failed sensor read publishes nothing
Presence	MQTT last-will message "offline" (retained, QoS 1) registered before the first connect; retained "online" birth message after every successful connect
Command intake	Subscribes to the device command topic on every (re)connect at QoS 1; exactly one supported command, LED SET_STATE with a boolean value (closed allowlist, not a generic passthrough)
Validation	Rejects malformed fields (MALFORMED), unsupported capability or command (UNSUPPORTED) and commands whose timestamp is more than 30 s from the device clock (STALE)
Acknowledgement	Publishes ACCEPTED after the output is driven, or REJECTED with a closed-enum reason; a command whose identifier cannot be determined gets no ACK because nothing could be correlated
Safe start	The LED output is set LOW before Wi-Fi or MQTT are started, so a reboot never leaves the actuator on
Not implemented	GPIO readback, current sensing, OTA update, self-registration, ACK retry, persistent command queue; relay, servo, fan and buzzer handlers and BME280/MQ-2 sampling are integration specifications, not firmware features

The distinction in the last row of the table is important for the reading of Chapter 4: the framework side of every capability (bindings, units, commands, ACK handling, readback mapping) is implemented and tested through the simulator, while the board-side drivers for the additional components are part of the integration specification.

3.5.7 MESSAGE FORMATS AND FAILURE HANDLING

The telemetry, status, command and ACK topics are those of Table 3.5. Telemetry carries the device identifier, an ISO-8601 UTC timestamp, the sensor type, a typed value and a unit. A command carries a command identifier, the capability, the command name, a boolean value and an epoch timestamp; an ACK echoes the command identifier with ACCEPTED or REJECTED and an optional reason. Examples are given in Appendix 4. The failure behaviour across device and backend is summarised in Table 3.18.

Table 3.18 Failure handling across the hardware path

Failure	Device behaviour	Backend behaviour
Wi-Fi lost	Reconnect loop; publish skipped while disconnected	No telemetry; Twin quality turns STALE after the freshness threshold
Broker connection lost	Last-will "offline" is delivered; reconnect and retained "online"	Connectivity set to OFFLINE; quality OFFLINE; OFFLINE_FOR timers can fire
Clock not synchronised	No publish, no command processing	No data; no invalid timestamp is stored
Sensor read fails	Nothing published that cycle	No update; quality ages normally
Malformed, unsupported or stale command	Rejected with a reason and not applied	ACK REJECTED recorded; command not treated as converged
Device reboot	Outputs return to the safe default	Desired state is not silently re-applied

Failure	Device behaviour	Backend behaviour
No ACK before the deadline	Not applicable	ACK timeout recorded; ACK-timeout trigger family available to rules
Publication failure	Not applicable	Publication status FAILED recorded; no convergence claim

3.5.8 VERIFICATION METHODOLOGY

Every claim about the hardware path is tied to a named evidence level, and a higher level is never inferred from a lower one (Table 3.19). The same chain is used for the simulator, which is why simulated evidence can be reported as software evidence and not as physical evidence.

Table 3.19 Evidence levels for a hardware-driven action

Level	Question answered	Evidence
E0	Does the circuit meet its electrical limits?	Multimeter readings of voltage, polarity and shared ground
E1	Does the firmware drive or read the pin locally?	Serial log and observation (for example LED blink)
E2	Does the broker receive the device message?	Broker capture of the telemetry topic
E3	Does NEXUS accept it?	Stored telemetry, Twin update and quality
E4	Does the device receive and acknowledge a command?	Correlated command, device log and ACK
E5	Did the physical effect occur?	Observation or independent measurement (LED, click, motion, current)

3.6 DATABASE DESIGN

The schema is created only by forward-only Flyway migrations (V1 to V39) and contains 42 tables (Figure 3.15). Telemetry history is a TimescaleDB hypertable with retention; events and pipeline traces are bounded by retention jobs. Historical migrations are never edited, and every migration is tested on a populated database as well as an empty one. Constraints carry the correctness rules: a unique plan per trigger fact or event, a unique candidate request id, check constraints on states and dispositions, foreign keys that restrict deletion of evidence, and a pair constraint that forbids a cooldown time without its attributed match.

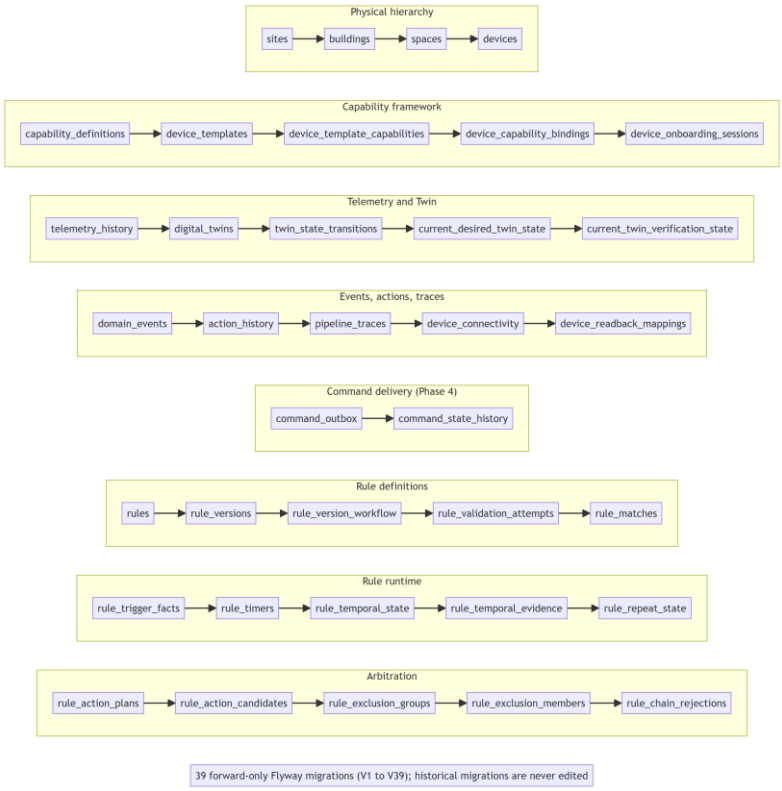


Figure 3.15 Database tables grouped by responsibility.

3.7 API DESIGN

The backend exposes 96 mapped endpoints in 26 controllers under `/api/v1`, all additive and backward compatible. Errors use a consistent structure with status, error, a stable machine code where the contract defines one, a message and a timestamp. Mutating endpoints that guard against concurrent edits take an expected revision and return a conflict when it is stale.

3.8 ARTIFICIAL-INTELLIGENCE SUBSYSTEM

3.8.1 PURPOSE AND DESIGN CONSTRAINTS

NEXUS contains one AI-assisted feature: explaining, in readable language, why a rule matched and what happened afterwards. The feature is deliberately narrow. The language model is never asked to decide, to activate a rule, to choose a command or to predict a physical effect. It receives a small, sanitised bundle of facts that the platform has already persisted and returns a structured explanation, and every statement in that explanation is then machine-checked against the same facts before anyone sees it. If anything fails, a deterministic renderer that uses no model produces the explanation instead. The design constraints that follow from this are listed in Table 3.20.

Table 3.20 Design constraints of the AI subsystem

Constraint	How it is enforced
The model cannot authorise or dispatch anything	The explanation package has no dependency on activation, arbitration or the action pipeline; the endpoint is read-only
The model sees only minimised data	A single mapper class decides what leaves the platform; match identifiers, rule identifiers and device targets are removed
Model output is untrusted	It is parsed into a strict structure, then grounding-checked claim by claim; the boundary returns text, never a finished explanation
Failure must never become a wrong answer	Five fixed outcome codes; every failure branch returns the deterministic explanation with a 200 response
The feature is optional and off by default	Configuration flag `nexus.ai.enabled` defaults to false; the platform works fully without any provider
Cost and abuse are bounded	Token-bucket rate limit per client address plus a global concurrency limit, both non-blocking
Physical effect is never asserted	Prompt rules, a platform-owned action note and a grounding rule that rejects claims about fields that do not exist

3.8.2 EVIDENCE CONTEXT

The explanation starts from a *context builder* that reads only persisted rows for one rule match: the match record with its rule snapshot and evaluation evidence, the rule revision and execution epoch, the triggering event, the persisted candidates of the action plan with their dispositions, reasons, winners and outcomes, the ActionHistory status with publication and acknowledgement data, and the persisted verification record where one exists. If a plan was never written (for example a legacy match), the context says so explicitly. Nothing is inferred at this stage.

3.8.3 DATA MINIMISATION

The context is turned into the payload for a provider by one small class, the context mapper, whose only job is to decide what may leave the platform. Table 3.21 shows the decisions. Because this is the single place where the decision is made, a reviewer can audit the whole data boundary by reading one file, and a unit test pins its behaviour.

Table 3.21 What the mapper sends to a provider

Item	Sent?	Note
Rule snapshot (name, conditions, actions, metadata, priority, enabled)	Yes	Needed to explain the match; the rule identifier and execution identity keys are removed
Evaluation evidence (condition, actual value, quality, result)	Yes	The values the FACT claims must cite
Match timestamp and message	Yes	
Actions (type and executor-recorded status)	Yes	Status is described to the model as a software record, not a physical confirmation

Item	Sent?	Note
Candidates (ordinal, action type, priority, disposition, reason, winner, outcome)	Yes	Targets are not included; publication and readback are reduced to presence flags
Rule revision and execution epoch	Yes	Version attribution
Match identifier, rule identifier, device identifiers, correlation identifiers	No	The model is told to emit a fixed placeholder match identifier, which the platform overwrites with the real one
Credentials and configuration	No	Never part of the context

3.8.4 PROMPT DESIGN

The request has two messages. The system message is written by the platform and fixed: it tells the model that it is a component inside an industrial IoT platform, that it must produce a structured explanation of the supplied evidence only, and it states the rules for the three claim tiers and the executor-status caveat. The user message contains the sanitised context serialised as plain JSON and nothing else. Because rule names, messages and evidence values are always sent as data in a separate message and never concatenated into the instruction, text inside a rule name that looks like an instruction is structurally treated as something to describe. The system message is only guidance; the safety guarantees are enforced afterwards by the parser and the grounding checker, not by trusting the model to follow the prompt.

3.8.5 PROVIDER BOUNDARY

The component that talks to a model implements a one-method interface that takes the sanitised context and returns an optional string of raw response text. It is not allowed to return an explanation object, so a provider cannot bypass the checks. The shipped implementation targets an OpenAI-compatible chat-completions endpoint and requests a JSON response that conforms to a schema. Timeouts, HTTP errors, refusals and empty responses all become an empty result. Provider-specific types are confined to one package, so another provider can be added by implementing the interface. The defaults, all overridable by environment variables, are shown in Table 3.22.

Table 3.22 AI configuration properties

Property	Default	Meaning
nexus.ai.enabled	false	Master switch; when false only the deterministic renderer runs
nexus.ai.provider	openai	Provider adapter selection
nexus.ai.base-url	https://api.openai.com/v1	Endpoint of an OpenAI-compatible API
nexus.ai.model	gpt-4o-mini	Model identifier sent in the request
nexus.ai.timeout-ms	8000	Per-request timeout

Property	Default	Meaning
nexus.ai.max-output-tokens	1024	Upper bound on the response length
nexus.ai.api-key	(empty)	Provided only through the environment; never committed
nexus.ai.rate-limit.requests-per-minute	5	Refill rate of the per-client token bucket
nexus.ai.rate-limit.burst	5	Capacity of the token bucket
nexus.ai.rate-limit.concurrency-limit	4	Global number of simultaneous explanations

3.8.6 STRICT PARSING AND CLAIM TIERS

Raw text from a model is first parsed by a strict candidate parser that requires the expected JSON structure and rejects anything else. Each claim has a tier (Table 3.23). The parser is also responsible for a subtle distinction that a Java record cannot express: a FACT claim whose JSON omits the asserted-value key is rejected, while one that sets it explicitly to null is accepted because the evidence may legitimately resolve to null.

Table 3.23 Claim tiers and their shape rules

Tier	Required	Forbidden	Meaning
FACT	evidence path and asserted value	supporting paths	One value directly present in the evidence
INFERENCE	one or more supporting paths	evidence path, asserted value	A reasoned connection between facts, never disguised as an observation
UNKNOWN	Nothing beyond the text	path, value, supporting paths	Something that cannot be established from the evidence

3.8.7 GROUNDING CHECK

The grounding checker is a deterministic function of a context and a candidate explanation, independent of how the candidate was produced, so it validates the renderer output and model output with the same code. For every claim it applies the following steps.

1. Reject the whole candidate if its match identifier does not equal the one of the context.
2. For a FACT claim, resolve the cited dot-and-bracket path inside the evidence; reject the claim if the path does not exist or if the value found there is not equal to the asserted value. Numeric strings are compared with numeric evidence by value, and booleans by boolean value.
3. For an INFERENCE claim, require every supporting path to resolve.

4. For an UNKNOWN claim, require that no path, value or supporting path is present.
5. Reject a claim with blank text. Rejected claims are never silently downgraded to UNKNOWN, so wrong text is never shown under a softer label.
6. Replace the model-supplied action note with the exact platform-owned disclaimer, and append the mandatory platform claims: causal uncertainty is always stated, and an unknown device is stated when no device identifier exists.

This guarantee is structural. It proves that each asserted value is really present in the evidence that the claim cites. It does not prove that the free-text sentence describing the value is a faithful paraphrase, and the report does not claim that it does.

3.8.8 FALLBACK AND OUTCOME CODES

The service that orchestrates all of this returns an outcome object that carries the explanation and one of five fixed reason codes (Table 3.24). Every branch except a database failure returns an explanation, so an unreachable provider, a refusal or a malformed response degrades to the deterministic explanation rather than to an error.

Table 3.24 Outcome codes of the explanation service

Code	Condition	Result returned
AI_SUCCESS	Model reply parsed and at least one claim survived grounding	Validated model explanation (rejected claims dropped)
AI_DISABLED	The feature flag is false	Deterministic explanation; no data leaves the platform
AI_UNAVAILABLE	Timeout, HTTP error, refusal, empty reply or exception in the provider	Deterministic explanation
AI_PARSE_FAILURE	Reply is not valid against the strict structure	Deterministic explanation
AI_GROUNDING_REJECTION	Zero claims survived grounding	Deterministic explanation

3.8.9 ENDPOINT, RATE LIMITING AND METRICS

The only HTTP entry point is `GET /api/v1/ai/explanations/rule-match/{matchId}`. It is read-only: it writes no telemetry, creates no event, changes no rule and sends no command. An unknown match returns 404. Before doing any work it asks a rate limiter for permission. The limiter combines a per-client token bucket (five requests per minute with a burst of five by default) with one global semaphore (four concurrent explanations); a caller that cannot obtain both

immediately is rejected and never queued. The client identity is the raw remote address, and a forwarded-for header is deliberately not trusted because no trusted-proxy configuration exists, so a spoofed header could otherwise defeat the limit. The limiter is cost protection only; it is not authentication. Micrometer records outcome counts and latency using only the five fixed reason codes as tag values, never identifiers, so metric cardinality stays low.

3.8.10 DETERMINISTIC RENDERER AND EXECUTION TRUTHFULNESS

The fallback renderer builds the same structure from the context with no model involved. It states facts such as the value and quality that satisfied each condition, the revision and epoch that matched, the arbitration disposition of each candidate and the outcome recorded by guarded acceptance. It describes an executor note only for executors that actually ran, and reports publication, acknowledgement and persisted verification as separate statements. The rule that physical actuation is UNKNOWN is built into the renderer rather than the prompt, which is why it holds when AI is disabled.

3.9 CORE ALGORITHMS

3.9.1 ARBITRATION

Arbitration is a pure function of the candidates of one trigger batch and the exclusion memberships. Its steps are given in Listing 3.1. The total order used everywhere is priority, then rule identifier, then revision, then ordinal, so the result never depends on repository iteration order, on the grouping order or on input order.

Listing 3.1 Arbitration, in outline.

```
ORDER = (priority, ruleId, revision, ordinal)

for each candidate c in sorted(candidates, ORDER):
    if c has no target key:            -> SELECTED  (INDEPENDENT_TARGET)
    else                               -> bucket by c.targetKey

for each target bucket:
    classes = group the bucket by equivalenceKey
    if exactly one class:              -> all members SELECTED/COALESCED
    best = priority of the first candidate
    winners = classes whose priority == best
    if more than one winner class:     -> every member SUPPRESSED_CONFLICT
    else: the winner class is SELECTED (primary + COALESCED contributors),
          all others SUPPRESSED_PRIORITY (pointing at the primary)

if exclusion memberships exist:
    consider only surviving command candidates
    for each group: the smaller configured rank wins;
        an equal best rank suppresses every competing command (conflict)
```

```
merge causes over groups by a total order, independent of group ids
```

3.9.2 COOLDOWN, EDGE AND HYSTERESIS

Three small state machines suppress repeated firing. *Cooldown*: before a match is persisted the engine takes a row lock on the cooldown row for the rule and scope; if the last attributed match is younger than the cooldown the candidate is skipped, otherwise the match and the new timestamp are written in the same transaction. *Edge*: a rule with EDGE_TRUE fires on a false-to-true change and is re-armed only by a definite false observation, never by missing or non-GOOD data. *Hysteresis*: a numeric condition with an enter and a strictly separated exit threshold flips its latch only when the value crosses the corresponding threshold with GOOD quality; values between the thresholds keep the previous state. Table 3.25 gives an example for a temperature rule.

Table 3.25 Hysteresis example for a temperature condition (enter above 30, exit below 28)

Observation	Quality	Latch before	Latch after	Condition result
29.0	GOOD	OFF	OFF	False (below enter threshold)
31.0	GOOD	OFF	ON	True (enter)
29.0	GOOD	ON	ON	True (inside the band, latch kept)
no value	STALE	ON	ON	Not satisfied; latch unchanged
27.5	GOOD	ON	OFF	False (exit)

3.9.3 QUALIFIED READBACK VERIFICATION

When a WRITE binding has a mapped READ binding, a later observation of the mapped capability is compared with the desired value. A GOOD-quality observation equal to the desired value establishes CONVERGED; a GOOD-quality observation that differs is DIVERGED; when no qualifying observation arrives before the stored deadline the verification becomes TIMED_OUT; a write-only capability with no mapping stays UNVERIFIED. Deadlines are persisted, so a restart does not lose a pending verification. None of these states asserts physical actuation: they describe agreement between a reported reading and an intent.

3.10 OPERATIONAL FLOWS

3.10.1 RULE-DRIVEN COMMAND: A WORKED SEQUENCE

The following narrative traces one rule, a temperature rule that switches a fan on, through the system, and names the table where each step leaves evidence.

1. A temperature reading arrives on the telemetry topic. The service validates the device, the capability, the type and the unit, writes a row to `telemetry_history` and updates `digital_twins`; the change is recorded in `twin_state_transitions`.
2. The Twin publishes a change event; the event detector stores a row in `domain_events`, and a `rule_trigger_facts` row with a routing disposition is written for the typed trigger.
3. The rule engine opens one REPEATABLE READ snapshot, evaluates the ACTIVE versions subscribed to that trigger, applies quality, unit, temporal, edge, cooldown and hysteresis policies, and writes a `rule_matches` row for each match.
4. One candidate per action is created; arbitration produces dispositions; a `rule_action_plans` row and its `rule_action_candidates` rows are written before anything is executed.
5. Guarded acceptance verifies the pinned version and epoch and the currentness of the source, then the command path writes the desired state and an `action_history` row, and publishes the MQTT command.
6. The device acknowledges on the ACK topic; the ACK handler records receipt and status. A later readback observation updates `current_twin_verification_state`.
7. The Operations Center shows the pipeline trace, and the explanation endpoint can reconstruct the story from these rows.

3.10.2 ONBOARDING OF A NEW DEVICE

A new ESP32 is added by choosing a template in the UI, which provisions the device, bindings and an onboarding session in one transaction and returns the generated MQTT contract. The operator flashes the board with that identity. The board publishes an online status, which moves the session to WAITING_FOR_TELEMETRY; its first valid reading is stored, applied to the Twin and activates the device in the same transaction. An invalid first reading is recorded as a rejection without activation, and an expired session can be retried.

3.11 CONFIGURATION AND SECURITY POSTURE

Configuration is read from `application.yml` with environment overrides. The principal settings are the MQTT topic patterns (telemetry, status, command and acknowledgement), the Twin freshness threshold (default 120 seconds) and maximum future-timestamp skew (five minutes), retention periods (telemetry 30 days, pipeline traces 7 days, events 90 days, actions 90 days) and the AI properties above. Secrets such as the AI key and the firmware Wi-Fi credentials come from the environment or an ignored file and are not committed.

The current security posture is that of a trusted local prototype and is stated explicitly rather than implied: the REST API has no authentication or authorisation, the MQTT broker accepts anonymous connections, there is no transport encryption and the only rate limit is the one on the AI endpoint. These gaps are recorded as Phase 7 work. Until that work is done the system must be used only on a network the operator controls.

3.12 RELIABLE COMMAND DELIVERY (PHASE 4)

Phases 1 to 3 made a command *accepted* only after a persisted, authorised plan, but the publication itself still happened in the same request after the commit. A crash between the commit and the publish, a broker outage or a slow acknowledgement could therefore leave an accepted command that was never sent, or leave its outcome unknown. Phase 4 adds a **durable delivery obligation** for every accepted command, a **bounded dispatcher** that is the only component allowed to publish it, and an **append-only history** of what happened to it. The slices implemented and reported here are P4.0 (contract, ticket and acceptance ledger), P4.1 (the outbox schema, configuration and retention protection) and P4.2 (atomic acceptance and the dispatcher), together with the corrections that an independent forensic review required. Retries with backoff, persisted ACK timeouts, telemetry deduplication, backpressure and delivery observability are planned slices (P4.3 to P4.8) and are not claimed.

3.12.1 GUARANTEE AND NON-GUARANTEES

With the outbox enabled, an accepted command and its delivery obligation are persisted together in one transaction; delivery is bounded by a window and attempt limits; and a restart never silently erases an obligation. MQTT QoS 1 is at-least-once,

so **exactly-once delivery and exactly-once physical actuation are neither provided nor claimed**. The committed firmware keeps no record of command identifiers, so a duplicate delivery of a command inside its freshness window would be applied again (the Python simulator, by contrast, keeps a bounded identifier cache). The design therefore never resends an uncertain delivery.

3.12.2 PERSISTED MODEL

Table 3.26 Tables added by Phase 4 (migrations V37 to V39)

Table	Purpose	Principal database-enforced rules
command_outbox (V37, V38)	One delivery obligation per accepted command	Primary key equals the ActionHistory request identifier; topic must match the device; payload is a JSON object whose command identifier and timestamp match the columns; SHA-256 equals the hash of the stored text; delivery window at most 20 s; attempts between 0 and 5; state, lease, retry-time and terminal columns must be coherent; at most one IN_FLIGHT row per device (unique index); identity and payload columns are immutable; terminal rows are immutable; only legal state transitions; a row cannot be claimed after it expired
Device deletion guard (V38)	Protect delivery evidence	A device with any outbox row cannot be deleted (stable messages DEVICE HAS UNRESOLVED COMMANDS and DEVICE HAS COMMAND DELIVERY EVIDENCE); the application maps them to HTTP 409
command_state_history (V39)	Append-only, ordered story of one command	Primary key (command, sequence); UPDATE always refused; DELETE only together with the parent ActionHistory row, so history can never block retention; fixed vocabularies for facts and sources; backfill of legacy commands is conservative and creates no outbox row

The payload is built **once**, at acceptance, and stored as text with its SHA-256. Every attempt publishes exactly those bytes, with the same command identifier and timestamp, and the dispatcher verifies the hash before the attempt. The firmware rejects a command whose timestamp differs from its own clock by more than 30 seconds in either direction, so the delivery window is capped at 20 seconds (30 s minus a 10 s allowance for timestamp truncation, clock error and network delay) and defaults to 15 seconds.

3.12.3 ATOMIC ACCEPTANCE

When the feature flag is on, the existing acceptance transaction additionally inserts the outbox row and two history facts. ActionHistory, the desired Twin state and its transition, the verification state, the outbox row and the history facts therefore commit together or not at all; the device-row lock order of earlier phases is unchanged. A constraint that refuses the outbox row turns into a rejection of the command, and no intent is stored. Authorisation happens once, at acceptance: the

dispatcher never re-evaluates rules, capabilities or connectivity, so a command that was legitimately accepted still finishes delivery if its rule is paused afterwards.

3.12.4 STATE MACHINE

Table 3.27 Delivery states and permitted transitions

From	To	Trigger
(none)	QUEUED	Acceptance commit
QUEUED	IN_FLIGHT	Claim by a dispatcher worker (due, not expired, nothing older unresolved for the device, no uncertain predecessor hold)
IN_FLIGHT	PUBLISHED	The MQTT delivery token completed inside the window; ActionHistory becomes SUCCESS in the same transaction
IN_FLIGHT	DEAD_LETTER	Proven non-publication, uncertain outcome, late completion or lease loss
QUEUED	DEAD_LETTER	The delivery window ended before any attempt (EXPIRED)
IN_FLIGHT, QUEUED	RETRY_WAIT	Reserved for Phase P4.3; the schema supports it but no code path uses it yet

3.12.5 DISPATCHER: CLAIM, LEASE AND FENCING

A scheduled loop, nudged after every acceptance commit, selects the oldest due command of each device whose window has not ended and claims it in its own short transaction with FOR UPDATE SKIP LOCKED. The claim records a lease owner, a lease expiry and an incremented attempt counter before any network input or output. Publication happens on a bounded worker pool **outside any transaction**, waiting on the MQTT delivery token for at most the attempt timeout (3 s by default) instead of calling the blocking client method, which can wait about 15 seconds. The outcome is written by a fenced transaction that applies only if the row is still IN_FLIGHT for the same lease owner and attempt; a stale worker writes nothing. The outbox row is locked first and the ActionHistory row second; the dispatcher never locks the device row. All decisions about claim eligibility, lease expiry and delivery expiry compare against the **database clock**, so application instances with skewed clocks agree.

A command is never claimed while an older command of the same device is unresolved, and a unique index forbids two in-flight commands for one device. This yields a proven **claim order** per device; it is not a claim about the order in which a device receives or acknowledges messages.

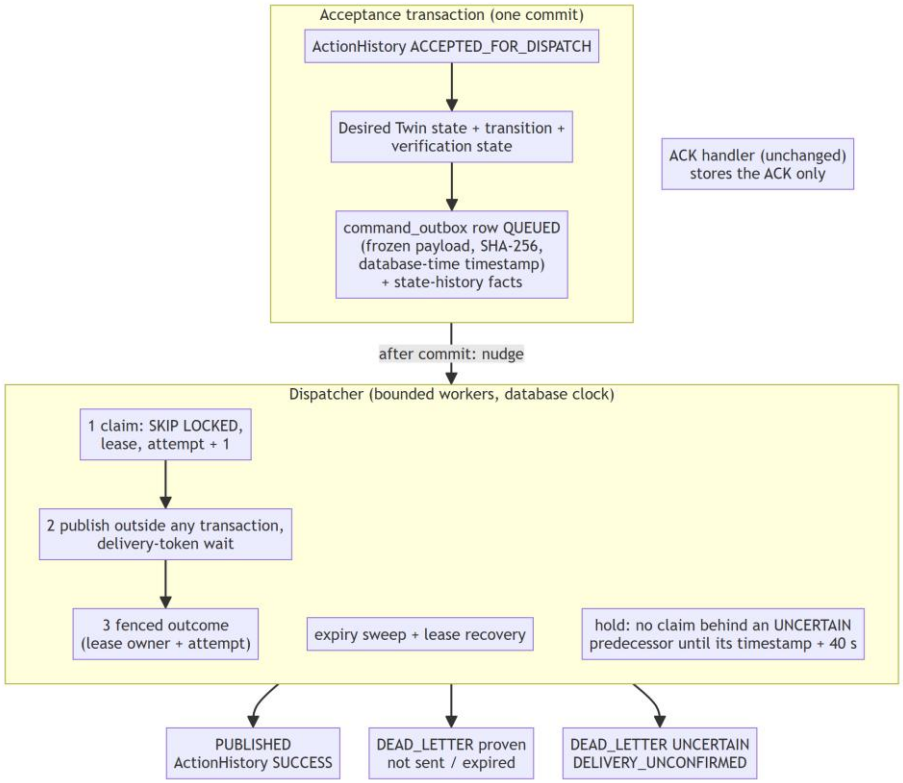


Figure 3.16 Phase 4 command delivery: one acceptance transaction, a fenced dispatcher and three honest outcomes.

3.12.6 CLASSIFICATION OF PUBLICATION OUTCOMES

Table 3.28 Publication outcomes and their recorded consequences (before retries exist)

Outcome	Evidence	Outbox / ActionHistory	Signals
Confirmed	The delivery token completed inside the window	PUBLISHED / SUCCESS with publication time and ACK deadline	Realtime SUCCESS, trace
Not sent	Explicit proof that the message never entered the client send path: client not connected, message or topic preparation failure, four listed pre-handover refusal codes, persistence refusal	DEAD_LETTER (permanent failure) / FAILURE "not sent"	Twin told once; one COMMAND_PUBLICATION_FAILED fact
Uncertain	Token timeout, connection loss after hand-over, any unlisted error, a confirmation recorded after the window, or a lost lease	DEAD_LETTER (uncertain) / FAILURE with the message DELIVERY_UNCONFIRMED	No failure fact and no Twin failure; the trace and realtime message say UNCERTAIN
Expired	The window ended before any attempt	DEAD_LETTER (expired) / FAILURE, never sent	Twin told once; one failure fact
Integrity failure	Stored hash does not match the stored payload	DEAD_LETTER (payload integrity), nothing sent	Twin told once

A timed-out publish may still reach the broker: the client keeps the QoS 1 message and resends it on reconnect. A lease timeout likewise does not prove that nothing was published. Both are therefore recorded as **uncertain**, never as plain failures and never as successes, and they are never resent. If a device acknowledged

the command before the outcome was recorded, every failure signal is suppressed because the device demonstrably received it.

3.12.7 UNCERTAIN-DELIVERY BARRIER

The firmware accepts any command that is fresh and has no identifier memory. If command B for a device were sent right after an uncertain command A, a late copy of A could be applied after B: an LED switched on by A and off by B could end up on while the desired state is off. NEXUS therefore holds later commands of that device until the uncertain command can no longer be applied: **its payload timestamp plus 40 seconds** (the 30-second firmware freshness limit plus the 10-second allowance), measured on the database clock and never longer than 60 seconds after its creation. The hold is derived from the persisted outbox rows, so it survives restarts and is identical for every instance; an ACK does not shorten it because a duplicate delivery would still be applied. A held command stays queued, is reported once in the state history, the ActionHistory message and the pipeline trace, and, if its own window ends first, is dead-lettered as expired and never sent. The price is that a command accepted right behind an uncertain one usually expires; this is visible, safe and documented, and no physical ordering guarantee beyond it is claimed.

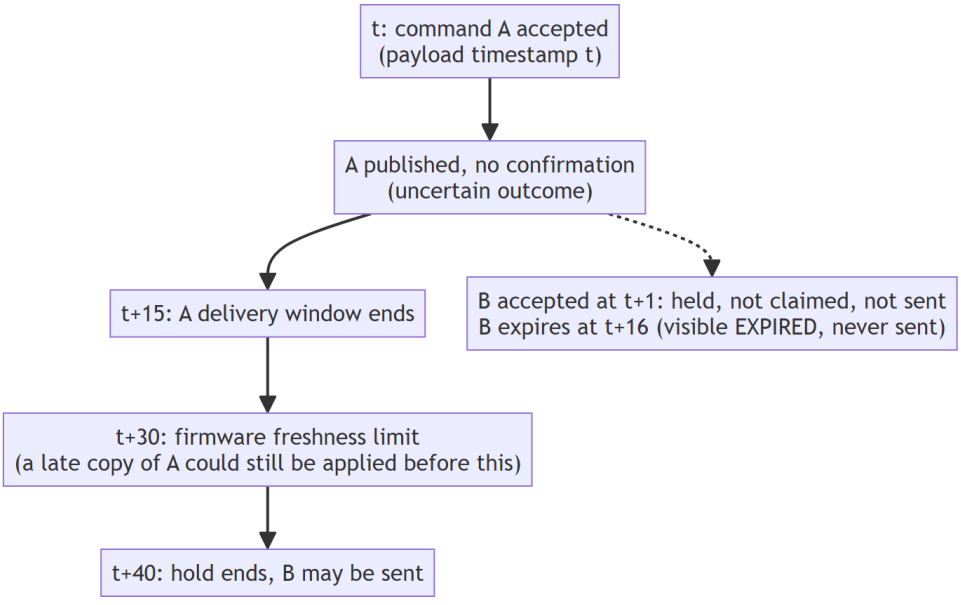


Figure 3.17 The uncertain-delivery hold relative to the firmware freshness limit.

3.12.8 FEATURE FLAG, ROLLBACK AND DATABASE TIME

The outbox is disabled by default. A single setting decides whether acceptance creates obligations, and it accepts only the exact words true and false; any other

spelling stops application start-up, because a looser conversion once allowed acceptance to enqueue while no dispatcher existed. The dispatcher is an unconditional component that never reads the flag, so turning the flag off again **drains** existing obligations (delivery, expiry, lease recovery) instead of stranding them, and start-up fails if the outbox is enabled without a dispatcher. In outbox mode the wire timestamp (still an integer of epoch seconds, the value the firmware compares with its own clock), the creation and scheduling times and therefore the expiry come from the database clock inside the acceptance transaction, so an application clock that is wrong in either direction cannot make a command unclaimable or immortal.

3.12.9 RETENTION AND DEVICE REMOVAL

Retention now runs each category (telemetry, traces, events, command outbox, action history) in its own transaction, so one failing category cannot roll back the others; it keeps action rows that current desired state or an outbox row still references and purges only resolved delivery records. Because a delivery record is evidence, a device that has any cannot be deleted until retention purges it (about 90 days); this fail-closed rule was introduced after a forensic experiment showed that deleting a device silently erased its pending obligations.

3.13 CORRECTNESS AND SAFETY PRINCIPLES

Table 3.29 Invariants enforced by the design

Invariant	Mechanism
A decision is persisted before any side effect	Plan and candidates are written in one transaction before guarded acceptance
One decision per trigger	Unique plan per fact or event; re-delivery reuses the plan
Stale or unsafe evidence cannot act	GOOD-quality and unit checks in the evaluator; source-currentness verdict in the snapshot
Configuration cannot race execution	Immutable versions, execution epoch, plan-bound authorisation
Conflicts fail closed	Equal-priority and equal-rank ties suppress all contenders
Command facts stay separate	Separate fields for desired, publication, ACK, readback, verification
Dry-run cannot write	Read-only REPEATABLE READ transaction; read-only collaborators; fingerprint tests
AI cannot authorise	Explanation component has no path to activation, arbitration or dispatch; grounding check on claims

3.14 TEST AND VALIDATION DESIGN

Verification is layered (Table 3.30). Unit tests cover pure logic such as arbitration and truthfulness. Integration tests run the full Spring application against a disposable

PostgreSQL (TimescaleDB image) and a real Mosquitto broker started by Testcontainers, so migrations, constraints, locks and the MQTT path are exercised for real. Concurrency tests use several transactions with in-process caches and locks disabled to imitate separate instances. Mutation checks temporarily break a guard and confirm that a test fails. The official gate combines repository tooling tests, backend and frontend suites, the simulator tests and three isolated smoke tests.

Table 3.30 Evidence layers

Layer	Examples	Infrastructure
Unit	Arbiter permutations, explanation truthfulness, workflow compiler	None
Integration	Rule lifecycle, temporal operators, dry-run API and equivalence, HTTP limit	Disposable PostgreSQL, Mosquitto
Migration	V24 to V36 on a populated database; checksums of V1 to V25	Disposable PostgreSQL
Concurrency	Two workers, one plan; two instances, one match; pool of two connections	Disposable PostgreSQL
Mutation	Remove a guard and expect a failure	Temporary source change
End-to-end smoke	Simulated loop; MQTT command and ACK; database restart	Isolated stack
Frontend	Typed forms, canvas save, simulator display	Vitest and jsdom

3.15 FEASIBILITY, RISKS AND ETHICS

The scope was feasible because it relies on open-source components, a laptop-scale environment and a simulation-first strategy that needs no special hardware. Table 3.31 lists the main risks and how they were managed.

Table 3.31 Principal risks and mitigations

Risk	Mitigation
Complexity growth in the rule engine	Contract-first slices, dependency-ordered ticket, acceptance ledger
Claims that exceed the evidence	COVERED only with exact test names; limitations recorded; independent forensic reviews
Tests damaging the development database	Disposable containers; scripts refuse to run against the live application
Machine resource limits (16 GB)	Memory-bounded test runs, sequential execution, bounded JVM settings
Security gaps	Documented: no authentication, anonymous broker for trusted local use; deferred to Phase 7
Misleading AI output	Advisory-only explanations, grounding check, deterministic fallback

Ethical considerations were handled as follows. No personal data is processed. Secrets are kept out of the repository and physical effects are never inferred from software facts.

Chapter 4
Implementation and Results

4.1 IMPLEMENTATION ENVIRONMENT AND CODE SIZE

Table 4.1 Implementation environment

Item	Version / detail
Operating system for development	Windows 11, 16 GB memory laptop
Backend runtime	Java 21, Spring Boot 3.3.4, Maven wrapper, Flyway
Database	PostgreSQL 16 with TimescaleDB 2.16.1 (container image timescale/timescaledb:2.16.1-pg16)
Broker	Eclipse Mosquitto 2.0 (container)
Frontend	Node.js 22 or later, React 19, TypeScript, Vite, React Query, Vitest
Simulator	Python 3.11 or later, paho-mqtt, pytest
Firmware toolchain	PlatformIO, ESP32 Arduino framework
Test infrastructure	JUnit 5, Mockito, Testcontainers, Docker Desktop

Table 4.2 Repository size at the final checkpoint

Component	Files	Approximate lines
Backend production code (Java)	368	21,300
Backend test code (Java)	155	29,500
Frontend source (TypeScript/TSX)	189	17,800
Frontend tests	63 files	533 tests
Simulator (Python)	30	2,400
Flyway migrations	39	V1 to V39
Firmware (C++)	2 sketches/projects	363 + 51

Test code is larger than production code in the backend, which reflects the evidence-first approach. Within the backend, the rule package is the largest (140 production files), followed by device (80) and twin (37).

4.2 DEVELOPMENT CHRONOLOGY

The repository history contains 109 commits between 27 July and 14 October 2026 (Figure 4.1). Table 4.3 summarises the milestones.

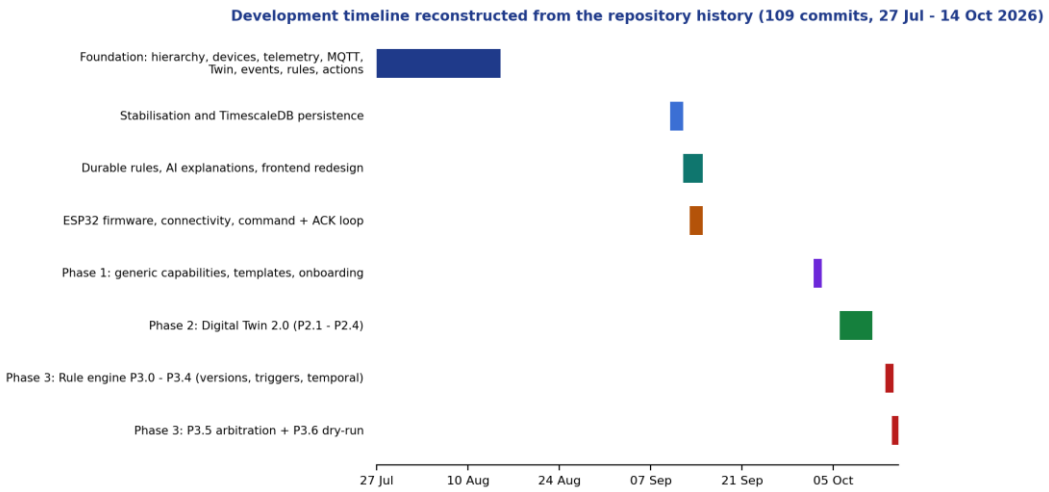


Figure 4.1 Development timeline derived from the commit history.

Table 4.3 Milestones

Period (2026)	Milestone
27 to 31 July	Foundation: hierarchy, devices, MQTT ingestion, telemetry, Twin v1, simulator
15 August	Phase 0 baseline and cleanup
10 to 13 September	Persistence, durable rules, AI explanations, firmware, command/ACK path
14 to 15 September	UI redesign; Review III simulated deployment (15 September)
2 to 3 October	Phase 1: generic capability framework, templates, onboarding
6 to 11 October	Phase 2: Digital Twin 2.0, quality, transitions, verification
13 to 14 October	Phase 3: lifecycle, typed triggers, temporal reasoning, arbitration, dry-run, grounded explanations

4.3 KEY IMPLEMENTATION DETAILS

4.3.1 ONE SNAPSHOT, ONE TRANSACTION

The most important implementation decision in the dry-run is that the whole request executes in one transaction. Listing 4.1 shows the annotation and its rationale as it appears in the source.

Listing 4.1 Transaction boundary of the dry-run service (excerpt).

```
/** ONE transaction for the whole request: REPEATABLE READ and read-only
 *  (so PostgreSQL itself refuses any write), on exactly ONE pooled connection.
 *  REQUIRES_NEW so an ambient read-write transaction is never joined. */
@Transactional(isolation = Isolation.REPEATABLE_READ,
               readOnly = true,
               propagation = Propagation.REQUIRES_NEW)
public DryRunResponse run(DryRunRequest request) {
    // everything is validated BEFORE any rule is evaluated
    if ("HISTORICAL_REPLAY".equals(mode))
        throw new DryRunRejection("HISTORICAL_REPLAY_UNSUPPORTED", ...,
                                  HttpStatus.UNPROCESSABLE_ENTITY);
    ...
}
```

4.3.2 BOUNDED REQUEST READING

A forensic review found that the request-size limit could be bypassed by a request path carrying a path parameter, because the limit was applied by a filter that matched the exact path. The corrected design reads the body inside the handler with a hard byte bound and refuses the first excess byte (Listing 4.2). An HTTP-level test matrix now exercises the path variants.

Listing 4.2 Bounded read in the dry-run controller.

```
static byte[] readBounded(InputStream in, int limit) throws IOException {
    var out = new ByteArrayOutputStream(Math.min(limit, 16 * 1024));
    byte[] buffer = new byte[8192];
    int total = 0, read;
    while ((read = in.read(buffer)) != -1) {
        total += read;
        if (total > limit)
            throw new DryRunRejection("INPUT_TOO_LARGE", "Dry-run input is
limited to "
                + limit + " bytes; nothing was evaluated.",
HttpStatus.PAYLOAD_TOO_LARGE);
        out.write(buffer, 0, read);
    }
    return out.toByteArray();
}
```

4.3.3 TRUTHFUL EXECUTION OUTCOMES

The explanation component never states that an action happened unless a persisted fact says so. For example, a command is described as ACCEPTED only if a desired-state transition exists for it, and an ActionHistory row without a candidate plan is reported as a fact about that row, not as proof that the arbiter selected it. The platform-owned note about executors is built only from the executors that ran for the match: a publication path for commands and a clearly labelled stub for alert and webhook executors.

4.4 OPERATIONS CENTER SCREENS

Figures 4.2 to 4.4 show three screens of the Operations Center. The screenshots were captured from the production build running in its opt-in offline demo mode, because the validation policy forbids starting the normal application against the development database. They therefore show empty or demonstration states and are included to illustrate layout and navigation, not live data.

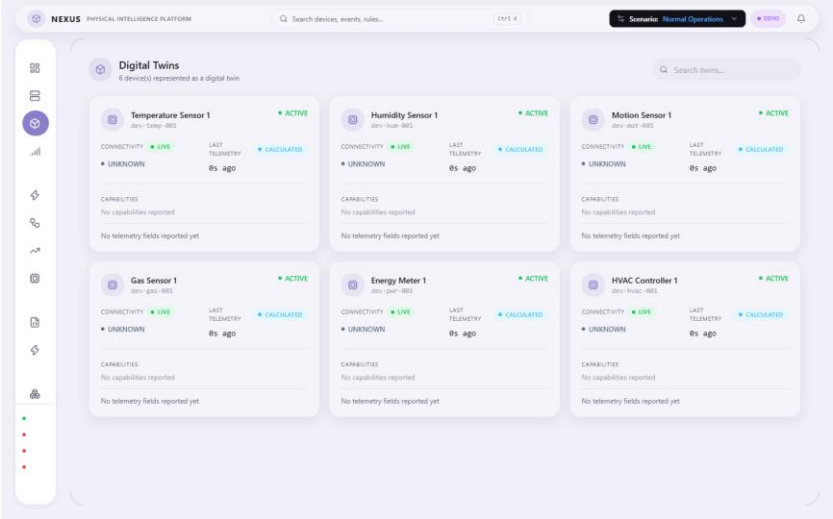


Figure 4.2 Digital Twins page (offline demo mode).

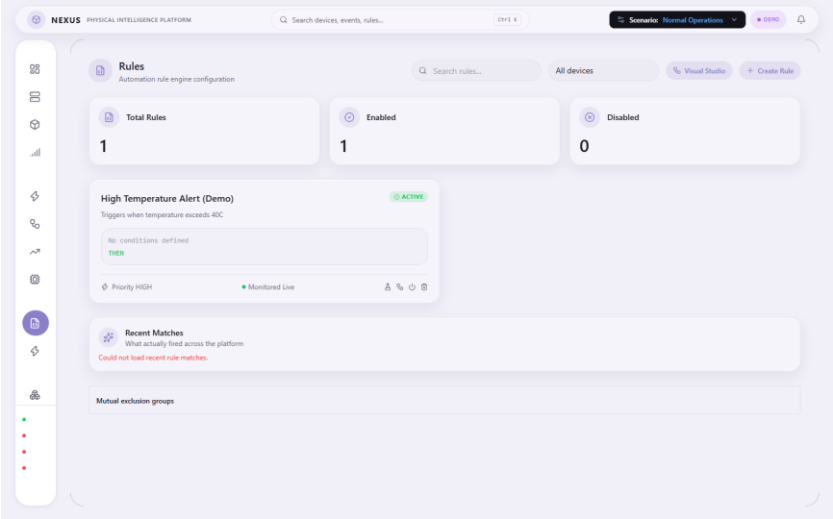


Figure 4.3 Rules page with the lifecycle and simulator entry points (offline demo mode).

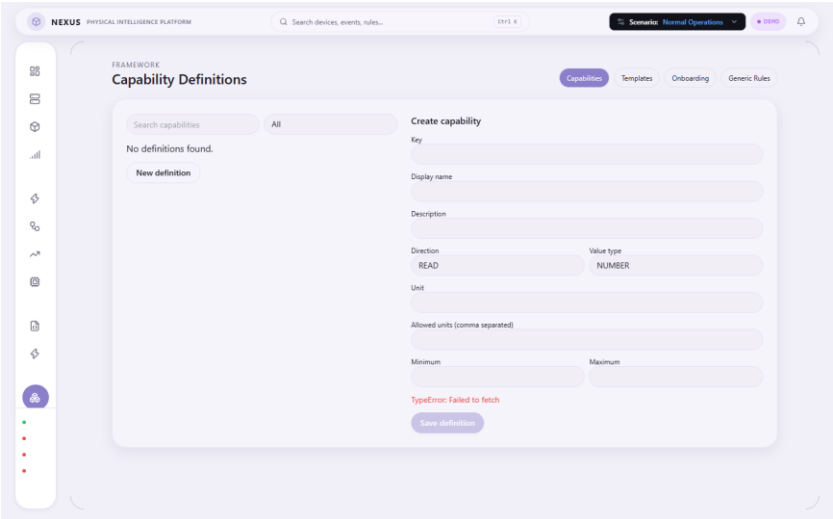


Figure 4.4 Capability framework administration page (offline demo mode).

4.5 TESTING AND VALIDATION

4.5.1 TEST INVENTORY

The final backend suite contains 181 test classes with 1,479 tests; the frontend contains 63 files with 533 tests. Test classes are concentrated where the risk is: the rule package has the largest number, followed by the action package (which holds the Phase 4 delivery tests), device and twin, with the remainder spread across infrastructure, integration, telemetry, event and other packages. Figure 4.5 and Table 4.4 show how the backend suite grew across successive checkpoints.

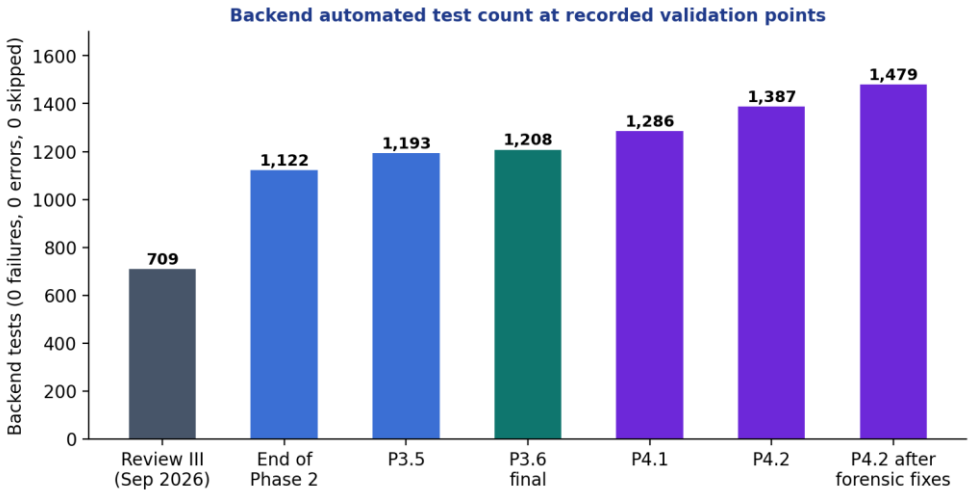


Figure 4.5 Backend automated test count at the recorded validation points.

Table 4.4 Backend test count at checkpoints

Checkpoint	Backend tests	Failures / errors / skipped
Review III (15 Sep 2026)	709	0 / 0 / 0
End of Phase 2	1,122	0 / 0 / 0
After P3.3 temporal reasoning	1,150	0 / 0 / 0
After P3.5 arbitration	1,193	0 / 0 / 0
After P3.6 and forensic corrections	1,208	0 / 0 / 0
After P4.1 (outbox schema, retention, deletion guard)	1,286	0 / 0 / 0
After P4.2 (atomic acceptance, dispatcher)	1,387	0 / 0 / 0
After P4.2 forensic corrections (final)	1,479	0 / 0 / 0

4.5.2 OFFICIAL VALIDATION GATE

The command `python validate_nexus.py --full` ran eleven checks in an isolated environment and passed 11 of 11 (Figure 4.6, Table 4.5). The fast gate passes 8 of 8. The backend step dominates the duration because it runs the entire integration suite against real containers.

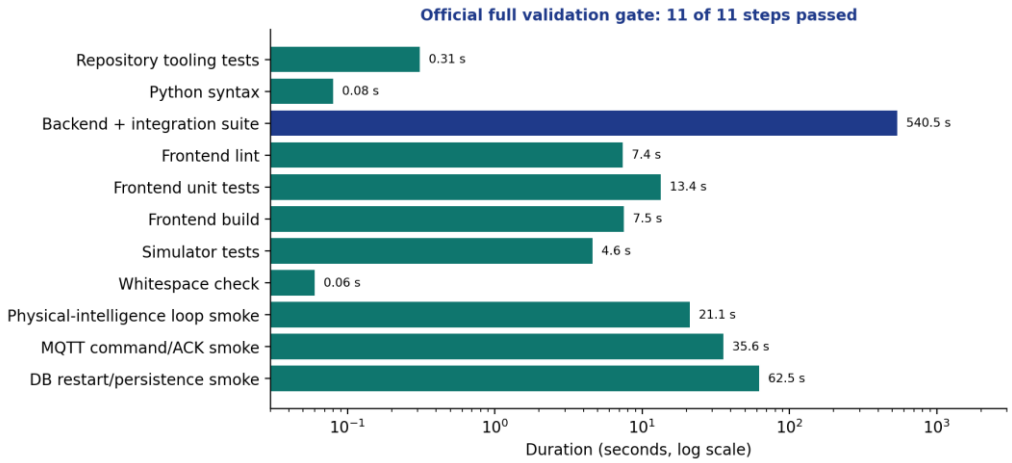


Figure 4.6 Duration of each full-gate check.

Table 4.5 Full validation gate

Check	Duration (s)	Result
Repository tooling tests	0.31	PASS
Python syntax	0.08	PASS
Backend unit and integration tests	540.5	PASS
Frontend lint	7.4	PASS
Frontend unit tests	13.4	PASS
Frontend production build	7.5	PASS
Simulator tests	4.6	PASS
Whitespace check	0.06	PASS
Simulated loop smoke test	21.1	PASS
MQTT command/ACK smoke test	35.6	PASS
Database restart smoke test	62.5	PASS

Frontend lint reports zero errors and twelve pre-existing warnings; TypeScript compilation and the production build are clean.

4.5.3 PHASE 3 ACCEPTANCE LEDGER

The Phase 3 ledger enumerates 72 scenarios; all 72 have status COVERED, none remain PLANNED (Appendix 1). COVERED means that exact automated tests exist and passed; it does not mean the feature is production ready. Open items recorded in the ledger are listed in the limitations section at the end of this chapter.

4.6 RESULTS

4.6.1 REVIEW III: SIMULATED THREE-BOARD DEPLOYMENT

On 15 September 2026 a simulated deployment of three virtual ESP32 boards ran for about 230 seconds against the then-current backend (709 tests). Table 4.6 lists the eleven scenarios and Table 4.7 the volume of data produced.

Table 4.6 Review III scenarios (expected versus observed)

Scenario	Expected behaviour	Result
Normal	Telemetry and Twin update, no rule match	PASS
High temperature	TEMPERATURE above 35 triggers FAN ON through the rule engine	PASS
Motion detected	MOTION true triggers LED ON (44 matches)	PASS
Gas alert	GAS above 400 triggers BUZZER ON	PASS
Gas + motion	GAS above 500 and MOTION true triggers buzzer and fan; above 700 also relay channel 1 (GAS 700.9, MOTION true observed)	PASS
Relay control	Three manual relay commands accepted through the real command path	PASS
Servo control	Three manual servo commands (0, 90, 180 degrees) accepted	PASS
Button press	BUTTON true triggers LED ON	PASS
Recovery	TEMPERATURE below 26 triggers FAN OFF	PASS
Multi-device isolation	Motion on a different device does not fire a device-scoped rule (43 to 43); on the right device it does (43 to 44)	PASS
Persistence and restart	Counts identical before and after a backend restart (devices 12, rules 9, matches 826, actions 893, traces 30,531)	PASS

Table 4.7 Data produced during the Review III run

Measure	Value
Telemetry records	+276
Domain events	+353
Rule matches	+132
Actions	+142
Pipeline traces	+694
Commands accepted	140

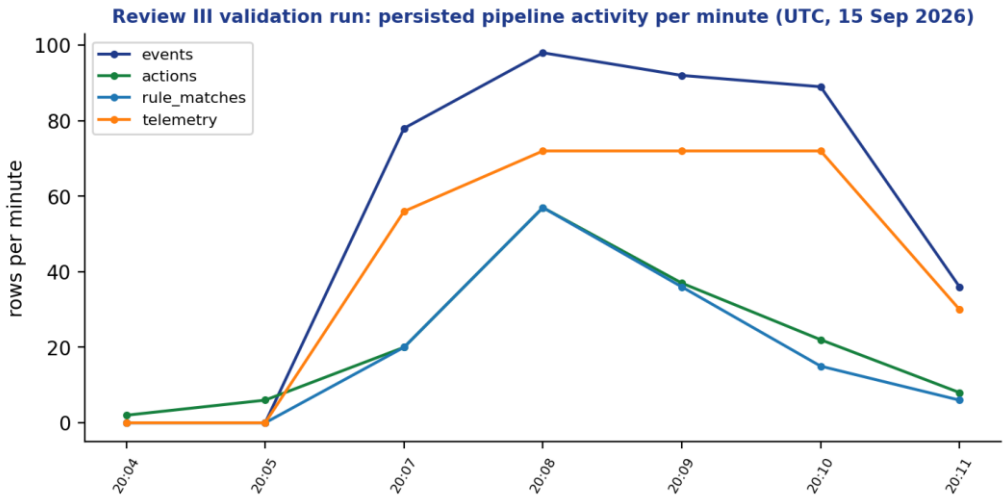


Figure 4.7 System activity per minute during the Review III run.

Command dispatch processing time inside the backend (n = 140) had a median of 5 ms, a 95th percentile of 49 ms and a maximum of 51 ms (mean 19.1 ms), shown in

Figure 4.8. This measures backend processing only; it is not a device round-trip time and says nothing about physical actuation latency.

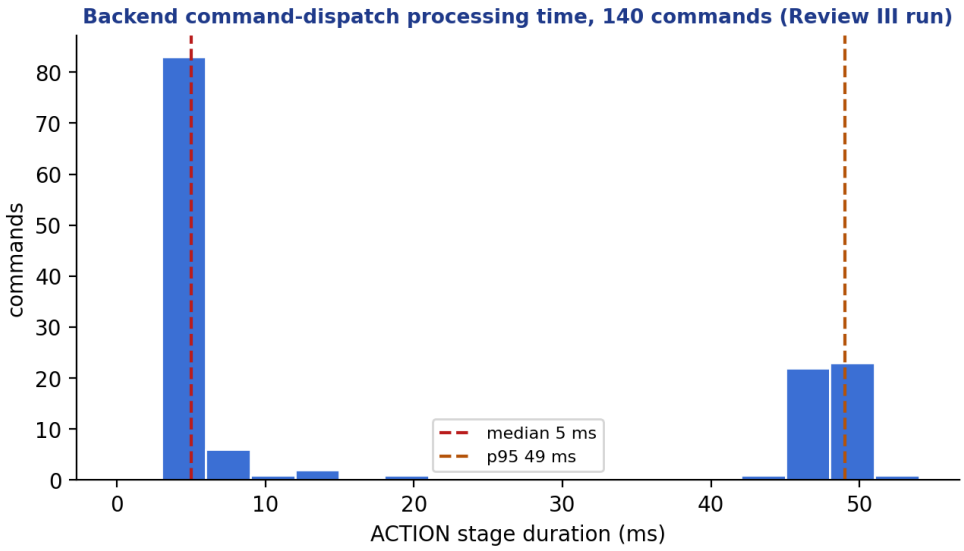


Figure 4.8 Distribution of command dispatch processing time (n = 140).

4.6.2 EQUIVALENCE, ZERO-EFFECT AND ROBUSTNESS EXPERIMENTS FOR THE DRY-RUN

Table 4.8 Experiments on the dry-run engine

Experiment	Method	Outcome
Production equivalence	Same inputs through production persistence and through the dry-run; compare matches, candidates, dispositions and winners	Identical for typed, temporal, edge, cooldown, hysteresis and exclusion cases
Zero effect	Fingerprint every relevant table before and after dry-run requests, including rejected ones	No row changed
HTTP limit matrix	Oversized bodies on exact, parameterised and trailing-slash paths	All refused with 413 before evaluation
Connection pool	Pool of two connections, concurrent requests	Initially starved; single-transaction design removed the starvation
Mutation checks	Temporarily remove a guard and run the guarding tests	Tests failed as intended; guards restored

4.6.3 FINDINGS FROM FORENSIC REVIEW

Independent review of the final slice reproduced and then corrected the defects summarised in Table 4.9. Each was first demonstrated by a failing test or experiment and then fixed.

Table 4.9 Defects found and corrected during final review

ID	Defect	Correction
B1	Request-size limit bypassed by a path parameter	Handler-side bounded read and HTTP matrix test

ID	Defect	Correction
B2	Connection-pool starvation from nested transactions	Single read-only REPEATABLE READ transaction
B3	Documentation claimed TREND was unsupported	Documentation corrected; TREND covered by scenario 49
B4	Temporal dry-run behaviour not proven equivalent	Equivalence tests added
B5	Edge re-arming not proven equivalent	EDGE_REARMED equivalence test
B6	Explanation note described executors that had not run	Note limited to executors that ran; verification claims persisted
B7	Inline definition accepted unknown fields	Strict inline definition
B8	Missing device configuration gave no diagnostic	DEVICE_NOT_CONFIGURED diagnostic
B9	Ownership and acceptance status unclear in output	Ownership fields and acceptanceAuthorization NOT_EVALUATED

4.7 HARDWARE INTEGRATION RESULTS

4.7.1 SOFTWARE-SIDE HARDWARE COMPATIBILITY

The hardware boundary is exercised in software at three levels. First, the simulator publishes telemetry, status and ACK messages in the same wire contract as the ESP32 firmware, and the telemetry and ACK formats of the two are kept identical by design. Second, the backend MQTT paths (telemetry listener, status handler, command publication and ACK handler) are tested against a real Mosquitto broker started by Testcontainers and, in the validation gate, by the isolated MQTT command and acknowledgement smoke test (35.6 s, PASS). Third, onboarding, quality, readback mapping and arbitration are tested for relay channels, servo angles, fans and buzzers through the simulator profiles. These results show that a board that follows the contract will be accepted by the framework.

4.7.2 ESP32 BENCH BRING-UP

The bench results that were actually observed are the following.

- **Serial interface.** An ESP32 board enumerated on the Windows development laptop as a Silicon Labs CP210x USB-to-UART adapter on COM5, establishing the programming and serial path.
- **Button input.** A pushbutton on GPIO33 with the internal pull-up (INPUT_PULLUP) was read correctly, a press being active LOW.
- **LED output.** An LED on GPIO26 blinked under firmware control, demonstrating firmware execution and GPIO switching on the board.

- **Buzzer driver finding.** An active buzzer with its positive lead directly on GPIO26 did not sound. This is consistent with the buzzer requiring more voltage or current than a GPIO can supply, and it fixed the design rule that buzzers, fans, servos and relay coils are driven through a transistor stage from a regulated supply.

Table 4.10 Hardware bench-test matrix and status

Step	Task	Status
H0	Record board models, breakout labels, ratings and power budget	Specification complete
H1	Flash ESP32 and confirm serial diagnostics	Serial link confirmed
H2	Button and LED local tests	Passed
H3	BME280, PIR and MQ-2 read separately	Protocol defined
H4	Power and driver circuits	Design defined; buzzer direct drive failed
H5	Each actuator driven locally	Protocol defined
H6	Real ESP32 telemetry accepted by NEXUS	Contract and software path validated with the simulator
H7	Command to ESP32 and ACK	Firmware path implemented for LED; software path validated
H8	Physical outcome verified	Protocol defined
H9	Offline, reconnect and negative cases	Firmware behaviour implemented; software cases validated
H10	End-to-end rule-driven demonstration	Protocol defined

4.7.3 PHYSICAL ACCEPTANCE-TEST PROTOCOL

Physical acceptance is defined by the protocol in Table 4.11. Each test names the procedure, the measurement and the pass criterion, and a test is recorded as passed only when its evidence is attached in Appendix 7. The protocol is part of the delivered verification methodology; it contains no results.

Table 4.11 Physical acceptance-test protocol

Test	Procedure	Pass criterion
PT1 Power	Measure supply voltage, polarity and common ground before energising each module	Within module ratings; no 5 V on any GPIO
PT2 BME280	Read at known ambient conditions for 5 minutes; compare with a reference thermometer	Values plausible, units correct, no zero substituted for a failed read
PT3 PIR	Walk in and out of view 10 times after warm-up	Motion true and false transitions logged with timestamps
PT4 MQ-2	Record a baseline after warm-up, then a safe low-level stimulus	Stable baseline; response direction correct; no direct 5 V on ADC
PT5 LED, buzzer	Drive each output from firmware through its driver	Observable and measured; boot state off
PT6 Servo, fan	Command three setpoints and on/off on a separate supply	Movement or rotation observed; current within supply rating
PT7 Relay	Command channels 1 to 4 individually on a low-voltage isolated load	Correct channel switches only; off at boot

Test	Procedure	Pass criterion
PT8 Telemetry	Run the node for 30 minutes with the broker capture on	Messages accepted; Twin quality GOOD; no rejected input
PT9 Command, ACK	Issue accepted, malformed, unsupported and stale commands	ACCEPTED or REJECTED with the correct reason and correct correlation
PT10 Fault	Disconnect Wi-Fi, stop the broker, reboot the board	OFFLINE quality; safe outputs after reboot; no unsafe actuation
PT11 End to end	Rule from sensor threshold to actuator on a low-voltage load	Complete evidence chain E0 to E5 with correlated identifiers

4.8 COMPARISON AND ANALYSIS

Table 4.12 compares NEXUS with the approaches reviewed in Chapter 2 along the dimensions that motivated the project. The comparison is qualitative and is based on the documentation cited in Chapter 2; the other systems are mature, widely deployed products with capabilities (scale, security, device management) that NEXUS does not attempt to match.

Table 4.12 Qualitative comparison

Dimension	Typical flow-based or home-automation engine	Cloud device shadow / twin service	NEXUS
Decision evidence persisted	Usually logs only	State documents	Match, plan, candidates, outcome
Stale-data safety	Usually application-defined	Timestamps provided	GOOD quality required to act
Conflict handling	Last write or order dependent	Not applicable	Deterministic arbitration, fail closed
Command facts	Often one success flag	Desired and reported	Five independent facts
Test before activation	Manual trial	Not applicable	Backend dry-run on the real engine
Operational maturity	High	High	Prototype

The analysis shows that the contribution is a coherent combination of known techniques (immutable versions, snapshot isolation, arbitration, readback verification) in a single traceable pipeline, together with a methodology that keeps claims within evidence. It is not a performance or scale contribution.

4.9 EVALUATION OF THE AI EXPLANATION SUBSYSTEM

Because the AI feature is the part of NEXUS most exposed to unpredictable behaviour, it received a dedicated evaluation. The tests use a fake provider that returns scripted replies, so every failure mode can be produced deterministically; no real model is called in the automated suite. Table 4.13 lists the 19 test classes (about 195 test methods) of the package and what each establishes.

Table 4.13 Test classes of the AI explanation package

Test class	Tests	What it establishes
GroundingCheckerTest	35	Every claim-tier shape rule, wrong and right asserted values, null handling, match identifier mismatch, forced platform disclaimer and mandatory claims
AiCandidateParserTest	16	Strict structure, missing and null asserted values, malformed JSON
OpenAiReasoningBoundaryTest	18	Request shape, timeout, HTTP error, refusal and empty response handling
PersistedArbitrationExplanationTest	18	Explanations state arbitration facts only when persisted
RuleMatchExplanationServiceTest	18	Each fallback branch and reason code
AiExplanationControllerTest	14	Status codes, rate-limit mapping, 404 for unknown match
ExplanationRateLimiterTest	13	Token bucket refill, burst, concurrency, deterministic clock
DeterministicExplanationRendererTest	11	Renderer claims and truthfulness
RuleMatchExplanationContextBuilderTest	9	Context assembled from persisted rows only
AiExplanationMetricsTest	8	Low-cardinality tags
AiExplanationControllerIntegrationTest	8	HTTP behaviour on real PostgreSQL
RuleMatchExplanationIntegrationTest	6	Explanation from real persisted matches
AiExplanationContextMapperTest	5	Identifiers removed, presence flags only
AiReasoningBoundaryEndToEndTest	4	Provider stub through parser and grounding
FailureContainmentEvaluationTest	3	Provider failures never reach the caller as errors
RuleExplanationPersistedFactsIntegrationTest	3	Persisted verification and acknowledgement claims
RuleMatchExplanationResponseTest	3	Response mapping
AiConfigurationTest	2	Defaults and disabled-by-default
RuleMatchExplanationEvaluationTest	1	Scenario library evaluation

The scenarios exercised include the following adversarial cases, each of which must end in either a rejected claim or the deterministic fallback and never in a displayed falsehood:

- a FACT claim whose asserted value differs from the evidence at the cited path;
- a FACT claim that cites a path that does not exist, such as an invented physical-outcome field;
- an UNKNOWN or INFERENCE claim that carries fields its tier forbids, or null lists that could throw an exception;
- a model-supplied action note that states a physical result, which is replaced by the platform disclaimer;
- a reply for a different match identifier;
- a provider that times out, returns an HTTP error, refuses or returns an empty body;

- a flood of requests from one address, which must be rate limited without blocking.

All of these scenarios pass. The evaluation is limited in a way that must be stated: it shows that the structural guarantees hold under a scripted provider, and it does not measure how often a real model produces useful, faithful sentences, how its latency varies or how it behaves on unusual rule definitions. Those require a live-provider study, which was not performed and is listed as future work.

4.10 SELECTED TEST SCENARIOS

Table 4.14 lists every third scenario of the Phase 3 acceptance ledger (the complete ledger is in Appendix 1), to show the breadth of behaviour covered. Each scenario is bound to named automated tests in the repository, and its status is COVERED.

Table 4.14 Sample of Phase 3 ledger scenarios

No.	Scenario	Evidence layer
1	Missing device/binding rejected	Unit + PostgreSQL activation
4	Invalid enum/Boolean/numeric value rejected	Unit boundaries
7	Re-enable reruns current validation	API + PostgreSQL
10	Activation selects validated version	PostgreSQL ownership
13	Archive is terminal	API + database
16	Old match survives edits/archive	History + explanation
19	Restart restores active ownership	Fresh Spring context + PostgreSQL
22	TRIGGER_DEVICE resolves one device	Scope tests
25	Accepted newer telemetry triggers	Producer + PostgreSQL
28	Replayed/out-of-order/future-rejected telemetry does not trigger	Ingestion/ordering integration
31	Publication failure differs from validation failure	Command integration
34	Verification timeout differs from ACK timeout	P2.4 transition integration
37	Schedule missed-run policy bounded	Clock + restart/multi-worker
40	Stale sample cannot prove duration	Quality gap tests
43	OFFLINE_FOR rejects UNKNOWN/reset on ONLINE	Connectivity + timer
46	Pending state survives restart safely	Fresh Spring context + PostgreSQL
49	TREND sample/span/change limits	Later temporal unit/integration
52	Identical commands coalesce with contributors	Plan + command integration
55	Cooldown survives versions/restart	Compatibility/history + durable gate integration
58	Hysteresis enter/exit boundaries	Controlled numeric tests
61	false/zero/valid empty STRING work	Backend + frontend
64	Dry-run temporal isolation	Runtime assertions
67	Explanation uses persisted facts	Grounding/fallback integration
70	Historical replay unsupported-mode validation	API + UI

4.11 DISCUSSION

Three observations follow from the results. First, the persist-before-act design makes a decision inspectable after the fact, because the rows that explain it already exist; this is what the explanation endpoint relies on. Second, concurrency defects were found only by tests that used several real transactions on a real database; the same logic passed single-threaded unit tests. Third, claims about physical effect are the easiest to overstate, and building the separation into the data model (five independent facts) and into the explanation checker made it difficult to overstate them accidentally.

There are also costs. The strict snapshot and persistence discipline add database writes per decision, and the full gate takes about nine minutes because of the integration suite. The rule engine is more complex than a flow-based engine for simple automations, which would be a poor trade for a user who only wants a few home-automation rules. The design targets settings where an unexplained or repeated physical action is costly.

4.12 RELIABLE DELIVERY VALIDATION (PHASE 4)

4.12.1 TEST EVIDENCE

The backend suite grew from 1,208 tests (end of Phase 3) to **1,479 tests in 181 classes**, with 0 failures, 0 errors and 0 skipped, on disposable PostgreSQL (TimescaleDB) and Mosquitto containers; the frontend (63 files, 533 tests) and the simulator (56 passed, 3 existing skips) are unchanged. Figure 4.5 shows the growth. Table 4.15 groups the Phase 4 tests by what they establish.

Table 4.15 Phase 4 test groups

Group	Layer	What it establishes
Schema and migrations	PostgreSQL; populated-database upgrade	Outbox and history constraints and triggers; device-deletion guard; V36 to latest upgrade leaves historical rows and checksums untouched; conservative legacy backfill
Atomic acceptance	Spring, PostgreSQL, Mosquitto, MockMvc	All records commit or none; an injected failure leaves nothing; a refused outbox row rejects the command; the API reports ACCEPTED_FOR_DISPATCH, never SUCCESS, for a queued command
Dispatcher	Real SQL with real connections; two instances	Claim, lease on the database clock, fencing, expiry, recovery of a lost lease, bounded workers, per-device claim order, no deadlock with acceptance, ACK handling and device removal
Uncertain-delivery barrier	PostgreSQL, two instances, restart	Hold boundaries against a model of the firmware freshness check, ACK does not shorten it, wrong clocks cannot lengthen it beyond the cap, held commands expire visibly

Group	Layer	What it establishes
Real ACK handler	Spring, PostgreSQL; concurrent	An ACK before, during and after the outcome, a rejected ACK racing an uncertain outcome, an ACK racing another acceptance, a stale worker after an ACK; nothing lost, overwritten or double-reported
Real wire	Real Mosquitto broker	One QoS 1, non-retained message whose bytes equal the stored payload; direct publication is refused
Flag and rollback	Spring; context runner	Only true or false accepted; dispatcher always present; rollback drains; restart recovery
Architecture audit	Source scan	Only the bounded publisher reaches the MQTT client; no requeue or resend operation exists; no unqualified exactly-once claim; delivery code never reauthorises
Compatibility suites	Real broker, flag off and on	The whole Phase 3 command-authorization suite (43 tests) and the dispatch suite (11 tests) pass unchanged with the flag off and, with isolated devices and queued-to-published assertions, with it on

4.12.2 INDEPENDENT FORENSIC REVIEW AND CORRECTIONS

As in earlier phases, each slice was reviewed adversarially before it was accepted, and the reviews found real defects, each reproduced by a failing test or experiment before being corrected (Table 4.16).

Table 4.16 Defects found in Phase 4 and their corrections

ID	Defect found	Correction
F1	Deleting a device cascaded to its outbox rows, silently erasing pending delivery obligations while the commands stayed accepted	Device key made RESTRICT; a database trigger refuses deletion while any delivery record exists; HTTP 409 with a stable code
F2	One protected action history row made the single retention transaction roll back every category	One transaction per category; desired-state and outbox reference guards; failures logged, counted and surfaced
H1	The dispatcher existed only for the exact property value true while the flag was converted with looser boolean rules, and turning the flag off stranded obligations	One strictly parsed authoritative flag; unconditional dispatcher; drain-only rollback; start-up validation
H2	A timed-out publish could be delivered later and overwrite a newer command	The uncertain-delivery barrier
M1	Any unexpected hand-over exception was classified as not sent	Only proven pre-handover refusals are not sent; everything else is uncertain
M2	An uncertain outcome raised a publication-failed fact and told the Twin the command failed, even after the device acknowledged it	No failure signal for uncertainty or an acknowledged command
M3	Creation and scheduling times came from the application clock but were compared with the database clock	Database time for the wire timestamp, creation, scheduling and expiry
Test	An ACK-handler test existed only as an SQL imitation; flag-on runs of legacy suites failed on delete-everything fixtures	Seven tests with the actual handler; isolated per-test devices and dual-mode assertions

4.12.3 MUTATION CHECKS

Seventeen temporary mutations of the new code were each shown to fail at least one test and were then reverted, with restoration verified byte for byte. They include removing the lease-owner and attempt fence, ignoring the delivery window or the

ordering guard in the claim, propagating the single-in-flight race, recording a late completion as a normal publish, skipping the hash check, allowing direct publication, swallowing an outbox failure during acceptance, removing the uncertain-delivery hold, classifying an unexpected error as not sent, emitting failure signals for uncertainty, taking the wire time from the application clock, gating the dispatcher on the flag and allowing it to be switched off while the outbox is enabled.

4.12.4 ACCEPTANCE LEDGER AND VALIDATION GATE

The Phase 4 acceptance ledger defines **152 scenarios**. At the end of the reported work 62 are covered by named tests with inspected assertions, 89 are planned for later slices and one (a general domain-event outbox) is deliberately deferred. A planned scenario is never counted as covered. The final full validation gate (eleven checks, including the simulated loop, the isolated MQTT command and acknowledgement smoke test and the database restart smoke test) and the fast gate (eight checks) passed. One earlier full run of the corrected tree failed with two context-load errors in an unrelated test class; its cause was not proven, a test-only configuration change was made that stops idle dispatchers polling in cached test contexts, and the repeated run passed. The failure is recorded here rather than omitted.

4.13 LIMITATIONS AND THREATS TO VALIDITY

- **No physical actuation evidence.** All command evidence is software-level (publication, acknowledgement, simulated readback). Hardware bench work established the serial link, a button input and an LED output; the physical sensor-to-actuator acceptance protocol (Table 4.11) defines how the remaining board-level evidence is recorded.
- **Simulated workload.** Review III used virtual boards for most scenarios; the single physical firmware covers temperature only.
- **No authentication or authorisation** on the REST API, and the broker is anonymous; rate limiting is absent. The system must be run only on a trusted network.
- **No automatic retry yet.** Commands now have a durable outbox, a fenced dispatcher and dead-letter outcomes (Phase 4), but each command is attempted once: a proven non-publication or an uncertain delivery ends in a recorded dead letter and is not retried. Retries with backoff for proven-unsent

commands, persisted ACK timeouts and telemetry deduplication are planned slices (P4.3 to P4.8); an uncertain delivery will not be retried without a device-side idempotency mechanism, which the committed firmware lacks.

- **Rule engine open items:** historical replay, manual approval, branching rule graphs, cron and time-zone schedules, cross-batch global arbitration, hypothetical devices and dry-run simulation of temporal arming from telemetry. Two behaviours are recorded as limits: a late match is not retroactively arbitrated, and a decided command is not re-dispatched if its executor failed.
- **Scale.** Correctness under concurrency was tested with a few workers and two simulated instances; no large-scale load or soak test was run, and unlimited scalability is not claimed.
- **Internal validity.** Tests were written by the same team that wrote the code. Independent forensic reviews and mutation checks reduce, but do not remove, shared blind spots.
- **Latency measure.** The dispatch latency is backend processing time only.
- **Screenshots** in this report come from offline demo mode.

Chapter 5
Conclusion and Future Work

5.1 SUMMARY OF THE WORK

This project designed, implemented and validated NEXUS, a simulation-first cyber-physical software platform that turns device telemetry into persisted, explainable and safely arbitrated physical-world decisions. The work began with a three-board prototype and produced a configuration-driven framework with capability definitions, templates and an onboarding workflow (Phase 1), a persistent Digital Twin with quality, history and readback verification (Phase 2), and a persistence-gated rule engine with typed triggers, durable temporal reasoning, deterministic arbitration, a backend dry-run and evidence-grounded explanations (Phase 3), and a reliable command-delivery layer with a transactional outbox, a fenced dispatcher and an uncertain-delivery barrier (Phase 4, slices P4.0 to P4.2). An Operations Center with a visual rule studio, a device simulator that speaks the hardware contract, and an ESP32 firmware that implements the same contract complete the system.

The delivered system comprises more than 20,000 lines of backend production code, 17,800 lines of frontend code and 39 forward-only database migrations, verified by 1,479 backend tests in 181 classes and 533 frontend tests in 63 files. The official eleven-check validation gate passed 11 of 11, the fast gate 8 of 8, all 72 scenarios of the Phase 3 acceptance ledger and 62 of the 152 scenarios of the Phase 4 reliable-delivery ledger are covered by named tests, and the simulated three-board deployment passed all eleven scenarios.

5.2 ACHIEVED CONTRIBUTIONS

- **Configuration-driven device abstraction:** capabilities, templates and per-device bindings replace hard-coded sensor types, and a persistent onboarding workflow provisions independent devices.
- **Persistent Digital Twin and quality-aware telemetry:** reported, desired and verification state with freshness and connectivity quality, and a stored transition history.

- **Deterministic rule processing:** immutable rule versions with a lifecycle and execution epoch, thirteen typed trigger families and evaluation against a single frozen snapshot.
- **Durable temporal reasoning:** duration, count, absence, offline and trend operators with database-backed state, plus edge, cooldown and hysteresis policies.
- **Safe action arbitration:** conflicts resolved by a deterministic total order, mutual-exclusion groups and fail-closed ties, with every decision persisted before any side effect.
- **Evidence-based command handling:** publication, acknowledgement, qualified readback and physical actuation recorded as separate facts, with a closed command allowlist on the device.
- **Reliable command delivery:** a durable outbox written in the acceptance transaction, a fenced and bounded dispatcher, an append-only command history, honest outcomes (published, proven not sent, uncertain, expired) and a barrier that stops a later command from overtaking an uncertain one.
- **Backend dry-run simulation:** the production decision logic reused without side effects, with proven equivalence and zero-effect behaviour.
- **Visual rule configuration:** a rule studio whose save and simulate actions use the real backend.
- **Grounded explanations:** an AI-assisted explanation path whose every claim is machine-checked against persisted evidence, with a deterministic fallback; the AI component cannot authorise or dispatch.
- **A verification methodology:** contract, ticket, ledger, tests and forensic review, which found and corrected real defects before completion.

5.3 SCOPE OF THE VALIDATED RESULTS

The software features above are complete and were validated through automated tests on real PostgreSQL and Mosquitto containers and through the simulated multi-device deployment. The simulation environment is the experimental validation environment of the project. The ESP32 subsystem was brought up on the bench (serial link, button input, LED output) and its firmware implements the telemetry, presence,

command and acknowledgement contract; the physical sensor-to-actuator acceptance protocol of Section 4.7 defines how the remaining board-level evidence is collected. The current release is a trusted-network prototype: it has no authentication, authorisation or transport encryption.

5.4 INTEGRATION OF SOFTWARE, HARDWARE AND AI

The three layers of the project meet at two contracts: the MQTT wire contract between devices and the platform, and the persisted-evidence contract between the platform and the AI explanation component. Table 5.1 states, layer by layer, what is integrated and at which evidence level. The statement is deliberately explicit about evidence so that the report never claims more than was demonstrated.

Table 5.1 Integration of the software, hardware and AI layers

Layer	What is integrated	Evidence level
Software platform	Ingestion, quality, Digital Twin, rule engine, arbitration, guarded command acceptance, durable delivery with a fenced dispatcher, dry-run, Operations Center	Automated tests on real PostgreSQL and Mosquitto: 1,479 backend and 533 frontend tests, 72 and 62 covered ledger scenarios, eleven-check validation gate
Device contract and simulator	Three simulated ESP32 profiles that speak exactly the hardware topics, payloads and acknowledgement semantics	Simulated three-board deployment: all eleven scenarios passed; wire contract tested against a real broker
Hardware (ESP32)	Firmware implementing telemetry, presence, a closed command allowlist (LED state), freshness check and acknowledgement against the same contract; bill of materials, electrical safety plan and a bench protocol	Bench bring-up documented (serial link, button input, LED output). End-to-end physical measurements (live sensors, driver circuits, closed loop with the real backend) belong in the hardware evidence sheets of Appendix 7 and are recorded there only when captured
AI-assisted decision support	An advisory explanation path that supports the operator in understanding every rule decision and whose every claim is machine-checked against persisted evidence, with a deterministic fallback and no route to activation or dispatch	Tests with a scripted provider; a live-provider quality study is future work

5.5 LESSONS LEARNED

- Persisting a decision before acting turns debugging from guesswork into reading rows.
- Separating facts (accepted, published, acknowledged, converged, actuated) prevents optimistic status reporting.
- Concurrency defects appear only when tests use several real transactions against a real database; mocks hid them.
- Independent adversarial review found defects that the author's own tests had not (Table 4.9).

- A simulator that shares the device contract lets the whole software stack be proven before hardware is introduced.

5.6 FUTURE WORK

The enhancements below extend the validated scope; none is required for the present system to be complete as a software framework.

Table 5.2 Future enhancements

Area	Description
Security and access control	Authentication, role-based access, MQTT ACLs and TLS, per-device credentials, rate limiting beyond the AI endpoint, audit log
Reliable dispatch (remaining slices)	Retries with backoff for proven-unsent commands only (an uncertain delivery is never retried without device-side idempotency), persisted ACK timeouts, telemetry deduplication, backpressure and delivery observability (Phase 4 slices P4.3 to P4.8)
Operational scaling	Metrics and alerting, load and soak tests with many devices, multiple backend instances
Rule engine extensions	Historical replay, manual approval workflows, branching graphs, cron and time-zone schedules, dry-run of temporal arming from telemetry, cross-batch arbitration
Protocol adapters	Adapter SDK and additional protocols such as Modbus
Extended physical deployment	Execution of the physical acceptance protocol on all three boards, further board drivers (relay, servo, fan, buzzer, BME280, MQ-2, PIR) and independent feedback sensors for convergence
Safe AI assistance	Natural-language rule drafting that always yields a reviewable, dry-run-tested draft, a live-provider study of explanation quality, and the advisory Jev extension proposed in Section 5.7.1

5.7 PROPOSED EXTENSION

5.7.1 PROPOSED ADVISORY EXTENSION (JEV)

As an extension to the existing NEXUS physical intelligence framework, the integration of **Jev, a structured AI decision model developed by TypeSafe AI**, is proposed to introduce contextual risk assessment and intelligent decision support. While the existing NEXUS Rule Engine evaluates predefined conditions and executes deterministic actions, the proposed Jev-based advisory module would analyze combinations of sensor measurements, Digital Twin states, historical events and device operating conditions to identify potentially abnormal situations and assess their operational significance.

The proposed module would be positioned between the Digital Twin and the Operations Center as an independent advisory service. Validated sensor data from ESP32 devices, including temperature, humidity, gas concentration and motion

detection, would first be processed by the existing telemetry ingestion and Digital Twin components. The advisory service would then construct a structured context containing recent measurements, state changes, device-quality indicators and relevant rule outcomes. This context would be submitted to Jev for evaluation through its structured decision interface.

The integration would use three decision mechanisms: **Choice**, to classify the operational state as NORMAL, WARNING or CRITICAL; **Score**, to estimate risk severity on a predefined scale; and **Noul**, to estimate the probability that a situation requires operator attention. For example, an increasing gas sensor reading combined with elevated temperature and detected movement could receive a higher advisory risk classification than an isolated sensor fluctuation. The resulting classifications and confidence information would be displayed in the NEXUS Operations Center to support faster incident investigation and operator decision-making.

An important design principle is that Jev would operate strictly as an **advisory intelligence layer**. It would not directly activate relays, fans, buzzers or other physical actuators. All command authorization, conflict arbitration and device control would remain under the existing deterministic Rule Engine and guarded ActionPlan execution mechanism. This separation would preserve system safety, auditability and explainability while introducing additional context-aware intelligence.

The proposed extension would be evaluated initially through simulation-based experiments using representative normal, abnormal and critical operating scenarios. Its performance would be assessed using classification accuracy, critical-event recall, false-alarm rate, response latency, confidence calibration and agreement with predefined safety conditions. The evaluation would also verify that unavailable or uncertain AI responses do not interrupt deterministic automation and cannot bypass the established command authorization process.

This extension would enhance NEXUS from a rule-driven automation platform into a hybrid physical intelligence framework that combines deterministic control, persistent Digital Twins and AI-assisted contextual decision support. It would provide a foundation for further developments in intelligent incident prioritization, predictive monitoring and adaptive operational analysis.

5.8 PROPOSED ARCHITECTURAL EVOLUTION

The current design is a modular monolith with a single database and a single backend process. That was the right choice for correctness work, because one transaction boundary makes persist-before-act and snapshot isolation simple to guarantee. The changes below are proposals that extend the validated scope; each is listed with the property it would add and the property it puts at risk.

Table 5.3 Proposed architectural changes

Change	Motivation	Risk to manage
Retries for proven-unsent commands (the outbox and dispatcher already exist)	Bounded retries with backoff inside the delivery window without losing the persist-before-act guarantee	An uncertain delivery must never be retried without device-side idempotency; the committed firmware has no command-identifier memory
Adapter gateway with an adapter SDK	Bind non-MQTT equipment (for example Modbus) through the same capability contracts	Each adapter must report quality and units honestly
Authenticated edge: RBAC, MQTT ACLs and TLS, per-device credentials	Remove the trusted-network assumption	Operational burden of key management
Edge-local rule execution on a gateway	Keep safety rules working when the backend or network is down	Version distribution and consistency with the central ledger
Event streaming backbone between modules	Scale ingestion and decouple consumers	Ordering and exactly-once expectations must be restated
Independently deployable services along the existing module boundaries	Scale the rule engine and ingestion separately	Loss of single-transaction snapshot; needs a new consistency design
Twin partitioning by site and read replicas	Large deployments	Cross-site rules
Observability stack (metrics, traces, alerting)	Operate and diagnose in production	Cardinality and cost

5.8.1 PROPOSED ADVISORY EXTENSION (JEV)

[Describe here the additional component or architecture change the team intends to add (referred to as JEV in the team plan): its purpose, where it sits in the architecture, which existing module it extends and how it would be validated. It is a placeholder because the repository contains no design for it, and this report does not describe a component that has not been specified.]

APPENDICES

Appendices give supplementary information that supports the report but would interrupt its flow. They are numbered using Arabic numerals. Tables and listings inside the appendices are numbered after the appendix to which they belong.

APPENDIX 1: ACCEPTANCE LEDGER OF PHASE 3

The ledger below is reproduced from the repository. Each scenario is COVERED by named automated tests; COVERED does not mean production ready.

Table A1.1 Phase 3 acceptance scenarios

No.	Scenario	Evidence layer	Status
1	Missing device/binding rejected	Unit + PostgreSQL activation	COVERED
2	Disabled binding rejected	PostgreSQL activation/runtime	COVERED
3	Wrong READ/WRITE direction rejected	Typed validator	COVERED
4	Invalid enum/Boolean/numeric value rejected	Unit boundaries	COVERED
5	Incompatible unit rejected	Unit + binding integration	COVERED
6	Invalid action type/parameters rejected	Validator + API	COVERED
7	Re-enable reruns current validation	API + PostgreSQL	COVERED
8	Failed activation changes nothing	Transaction rollback	COVERED
9	Draft validates a specific version	PostgreSQL lifecycle	COVERED
10	Activation selects validated version	PostgreSQL ownership	COVERED
11	Pause prevents new dispatch	Matcher/planner + guarded acceptance integration	COVERED
12	Resume revalidates and resets temporal progress	Lifecycle + runtime	COVERED
13	Archive is terminal	API + database	COVERED
14	Active edit creates successor draft	Version integration	COVERED
15	Legacy enabled PUT performs composite activation	API compatibility	COVERED
16	Old match survives edits/archive	History + explanation	COVERED
17	Concurrent edits with same revision token conflict	PostgreSQL barriers	COVERED
18	Activation/evaluation sees one pinned version	Deterministic concurrency	COVERED
19	Restart restores active ownership	Fresh Spring context + PostgreSQL	COVERED
20	Snapshot remains frozen during concurrent telemetry	PostgreSQL snapshot race	COVERED
21	DEVICE scope ignores unrelated device	Router/evaluator	COVERED
22	TRIGGER_DEVICE resolves one device	Scope tests	COVERED
23	Action target may differ from scope	Generic command integration	COVERED
24	Ambiguous legacy scope diagnosed	Populated migration preflight	COVERED
25	Accepted newer telemetry triggers	Producer + PostgreSQL	COVERED
26	Same-value telemetry is not semantic change	PostgreSQL transition counts	COVERED
27	Semantic change/creation triggers once	Transition identity integration	COVERED
28	Replayed/out-of-order/future-rejected telemetry does not trigger	Ingestion/ordering integration	COVERED
29	Legacy multiple-event behavior remains	EventDetector + matcher	COVERED
30	Connectivity duplicates do not trigger	Status transaction/concurrency	COVERED
31	Publication failure differs from validation failure	Command integration	COVERED
32	First ACK rejection triggers; duplicates do not	Handler + real Mosquitto	COVERED
33	ACK deadline equality does not time out	Controlled Clock	COVERED
34	Verification timeout differs from ACK timeout	P2.4 transition integration	COVERED
35	Convergence/divergence stay qualified readback facts	P2.4 regression	COVERED
36	Manual evaluation has truthful trigger/scope	API + persistence	COVERED
37	Schedule missed-run policy bounded	Clock + restart/multi-worker	COVERED
38	Rule-chain cycle/depth limits terminate	Router tests	COVERED
39	FOR_DURATION exact boundary	Controlled Clock + PostgreSQL	COVERED

No.	Scenario	Evidence layer	Status
40	Stale sample cannot prove duration	Quality gap tests	COVERED
41	COUNT_WITHIN window/counting mode	Unit + runtime integration	COVERED
42	ABSENCE_FOR ignores invalid/replayed data	Producer + timer	COVERED
43	OFFLINE_FOR rejects UNKNOWN/reset on ONLINE	Connectivity + timer	COVERED
44	Duplicates do not advance temporal state	Runtime identity tests	COVERED
45	Late/out-of-order evidence does not rewind	Temporal integration	COVERED
46	Pending state survives restart safely	Fresh Spring context + PostgreSQL	COVERED
47	Timer/telemetry race coherent	Forced PostgreSQL lock orders	COVERED
48	New version does not inherit temporal progress	Lifecycle/runtime	COVERED
49	TREND sample/span/change limits	Later temporal unit/integration	COVERED
50	Smaller numeric priority wins	Arbitration + PostgreSQL plan	COVERED
51	Equal-priority conflict suppresses	Arbitration evidence	COVERED
52	Identical commands coalesce with contributors	Plan + command integration	COVERED
53	Relay channel isolation	Command integration	COVERED
54	Different devices independent	Canonical target tests	COVERED
55	Cooldown survives versions/restart	Compatibility/history + durable gate integration	COVERED
56	Concurrent cooldown checks allow one match	Multi-processor PostgreSQL race	COVERED
57	Mutual exclusion/safety policy explicit	Arbitration integration	COVERED
58	Hysteresis enter/exit boundaries	Controlled numeric tests	COVERED
59	Live/dry decisions equivalent	Shared-context comparison	COVERED
60	Dry-run zero production effects	API + DB/MQTT/realtime spies	COVERED
61	false/zero/valid empty STRING work	Backend + frontend	COVERED
62	Supplied-state limits/types enforced	API validation	COVERED
63	Current-state mode uses backend state	API + snapshot	COVERED
64	Dry-run temporal isolation	Runtime assertions	COVERED
65	Dry-run returns arbitration evidence	API + frontend	COVERED
66	Legacy and generic rules supported	Existing regression + new adapters	COVERED
67	Explanation uses persisted facts	Grounding/fallback integration	COVERED
68	Rule -> command -> ACK -> echo -> convergence	Real Mosquitto + PostgreSQL	COVERED
69	Unsupported manual approval cannot dispatch	Feature validation + planner	COVERED
70	Historical replay unsupported-mode validation	API + UI	COVERED
71	Populated migration preserves history	Real populated V24 -> V25	COVERED
72	Source/event persistence failure blocks dispatch	Failure-injection integration	COVERED

APPENDIX 2: PHASE 4 ACCEPTANCE LEDGER SUMMARY

The Phase 4 ledger owns the testable scenarios of reliable command and event delivery. Table A2.1 summarises it by slice. Scenarios of later slices are planned and are not claimed.

Table A2.1 Phase 4 ledger status by slice

Slice	Covered	Planned	Deferred	Subject
P4.1	17	0	0	Outbox schema, configuration, retention protection, device-deletion guard
P4.2	45	1	0	Atomic acceptance, dispatcher, state history, safety corrections
P4.3	0	17	0	Retries, expiry boundary, duplicate behaviour
P4.4	0	14	0	ACK facts, persisted timeout, legacy reconciler
P4.5	0	17	0	Telemetry deduplication
P4.6	0	6	0	Backpressure and rate limiting
P4.7	0	14	0	Observability, read API and frontend
P4.8	0	20	0	Integrated validation and enablement
none	0	0	1	General domain-event outbox (deferred)

APPENDIX 3: REPOSITORY STRUCTURE

The main directories of the project repository are listed below. Generated output, local runtime state and secrets are not tracked.

Listing A3.1 Main directories of the repository.

```
nexus/
  backend/         Spring Boot 3 modular monolith (Java 21), Flyway migrations
V1-V39
    com.nexus.device      registry, capabilities, templates, onboarding,
commands
    com.nexus.telemetry   validation and typed history
    com.nexus.twin        Digital Twin state, quality, readback, verification
    com.nexus.event       domain events and dispatcher
    com.nexus.rule        versions, triggers, plans, dry-run, AI explanations
    com.nexus.action      executors, action history, command outbox, dispatcher
    com.nexus.pipeline, realtime, analytics, infrastructure (MQTT)
  frontend/        React 19 Operations Center (Vite, TypeScript, React Query)
  simulator/       Python device simulator and provisioning scripts
  firmware/        ESP32 firmware (telemetry, presence, command, ACK)
  hardware-tests/  bench protocols and evidence sheets
  infrastructure/  Docker Compose, Mosquitto configuration
  architecture/    contracts, tickets and acceptance ledgers (phases 2-4)
  docs/, tests/    project documentation and cross-component tests
  start_nexus.py   orchestrator for the complete stack
  validate_nexus.py  fast (8 checks) and full (11 checks) validation gate
```

APPENDIX 4: MQTT MESSAGE FORMATS

The messages below are examples of the wire formats used by the firmware, the simulator and the backend. Identifiers are examples.

Table A4.1 Message examples

Topic	Example payload
nexus/devices/{id}/telemetry	{"deviceId": "<uuid>", "timestamp": "2026-10-14T10:15:00Z", "sensorType": "TEMPERATURE_SENSOR", "value": 27.4, "unit": "CELSIUS"}
nexus/devices/{id}/commands/ack	{"commandId": "<uuid>", "status": "ACCEPTED", "timestamp": 1760436900}
nexus/devices/{id}/commands/ack (refusal)	{"commandId": "<uuid>", "status": "REJECTED", "timestamp": 1760436900, "reason": "<text>"}

APPENDIX 5: SELECTED SOURCE LISTINGS

The listings below are copied from the repository at the final checkpoint and shortened. They show the decision function that every rule match passes through and the device-side telemetry publication.

Listing A5.1 Rule decision function (RuleDecisionLogic.decide), shared by production and dry-run.

```
public static Decision decide(RuleEvaluator evaluator, Rule rule,
        RuleEvaluationContext context, boolean cooling, BooleanSupplier edgeLatch) {
    var captured = context.captured();
    boolean stateful = stateful(rule);
    if (cooling && !stateful)
        return new Decision(Outcome.SKIPPED_IN_COOLDOWN, null, List.of(), false,
List.of());
    if (captured.scopeMissing())
        return new Decision(Outcome.SCOPE_MISSING, null, List.of(), false,
List.of());
    if (!captured.diagnostics().isEmpty())
        return new Decision(Outcome.CONTRACT_DIAGNOSTICS, null, List.of(), false,
captured.diagnostics());

    RuleEvaluationResult result =
        evaluator.evaluate(rule, context.trigger().legacyEvent(), context);
    ...
    if (rule.repeat() == RepeatPolicy.EDGE_TRUE && scope != null && identity !=
null) {
        if (result.matched() && edgeLatch.getAsBoolean())
            return new Decision(Outcome.EDGE_SUPPRESSED, result, effects, false,
List.of());
        // Only definite false evidence rearms the edge.
        rearm = !result.matched() && definitelyFalse(result);
    }
    if (result.matched() && cooling)
        return new Decision(Outcome.COOLDOWN_SUPPRESSED, result, effects, rearm,
List.of());
    return new Decision(result.matched() ? Outcome.MATCHED : Outcome.NOT_MATCHED,
                        result, effects, rearm, List.of());
}
```

Listing A5.2 ESP32 firmware: telemetry publication with QoS 1 (firmware/src/main.cpp).

```
if (!isTimeSynced()) {
    Serial.println("[Time] Clock not yet synced; skipping this publish cycle ...");
    return;
}
JsonDocument doc;
doc["deviceId"]   = NEXUS_DEVICE_UUID;
doc["timestamp"]  = currentIsoTimestampUtc();
doc["sensorType"] = "TEMPERATURE_SENSOR";  // wire-level parity with the simulator
doc["value"]      = temperatureCelsius;
doc["unit"]       = "CELSIUS";
String payload;
serializeJson(doc, payload);
bool ok = mqttClient.publish(telemetryTopic, payload, /*retained=*/false,
/*qos=*/1);
```

APPENDIX 6: HOW TO RUN AND VALIDATE THE SYSTEM

The commands below are taken from the repository documentation. They assume Windows PowerShell, Docker with Compose, Java 21, Node.js 22 or later and Python 3.11 or later. Real keys must never be committed; copy `.env.example` to `.env` for local overrides.

Table A6.1 Operating commands

Purpose	Command
Start the complete stack with the simulator	python start_nexus.py
Start without the simulator	python start_nexus.py --no-simulator
Timed run, then stop infrastructure	python start_nexus.py --duration 60 --infra-down-on-exit
List or stop services	python stop_nexus.py --list ; python stop_nexus.py --infra
Fast validation (8 checks)	python validate_nexus.py --fast
Full validation (11 checks, application stopped)	python validate_nexus.py --full

Startup rejects occupied ports unless an independently running NEXUS service is explicitly reused, and a second live orchestrator is always rejected. Runtime ownership, logs and validation reports are stored in an ignored `.nexus` folder. Validation must be run with the application stopped, so it never touches the development database. To add a physical node, provision it from a template in the UI, flash the generated identity and contract onto the board, and confirm that status and first telemetry advance the onboarding session to OPERATIONAL.

Table A6.2 Commands used to validate the repository

Scope	Working directory	Command
Fast gate (8 checks)	Repository root	python validate_nexus.py --fast
Full gate (11 checks)	Repository root; normal app stopped, Docker available	python validate_nexus.py --full
Backend	backend/	./mvnw.cmd test
Backend, one class	backend/	./mvnw.cmd "-Dtest=ClassName" test
Frontend tests	frontend/	npm.cmd test -- --run
Frontend lint and build	frontend/	npm.cmd run lint; npm.cmd run build
Simulator	simulator/	./.venv/Scripts/python.exe -m pytest -q
Whitespace	Repository root	git diff --check

Final recorded results: full gate 11 of 11 passed; fast gate 8 of 8; backend 181 classes, 1,479 tests, 0 failures, 0 errors, 0 skipped; frontend 63 files, 533 tests; Phase 3 ledger 72 COVERED, 0 PLANNED; Phase 4 ledger 152 scenarios, 62 COVERED, 89 PLANNED, 1 DEFERRED.

APPENDIX 7: HARDWARE EVIDENCE SHEETS

This appendix is reserved for the photographs and logs that document each hardware bench step. Only evidence that was actually captured should be inserted; a step with no evidence must stay marked as not yet validated in Table 4.10.

Table A7.1 Evidence checklist

Step	Evidence to insert	Inserted?
H1	Photograph of the board on COM5 and the upload and boot serial log	**[yes / no]**
H2	Photograph of the GPIO33 button and GPIO26 LED circuit; serial log of presses	**[yes / no]**
H3	BME280, PIR and MQ-2 readings with timestamps and measured pin voltages	**[yes / no]**
H4 and H5	Driver circuit photographs; measured voltages; video of each actuator	**[yes / no]**
H6	Broker capture of real ESP32 telemetry and the matching Twin record	**[yes / no]**
H7 and H8	Correlated command, device receipt, ACK and the observed physical result	**[yes / no]**
H9 and H10	Offline, reconnect and end-to-end rule-driven demonstration	**[yes / no]**

APPENDIX 8: INDIVIDUAL CONTRIBUTION AND PUBLICATION STATUS

The guidelines require each team member to state their individual contribution. This is the statement of the author of this individual report, BALAVIGNESH R (23BCE1802); the NEXUS project was developed collaboratively and the other members state their own contributions in their reports.

Table A8.1 Division of work in the project team

Team member	Area	Main work
BALAVIGNESH R (23BCE1802)	Reliable decisions, command delivery and validation	Rule execution safety (guarded acceptance, arbitration, cooldown and hysteresis, dry-run). Phase 4 command outbox, single fenced dispatcher, uncertain-delivery hold and append-only command state history. Acceptance ledgers, dual-mode test suites and validation gates. ESP32 bench work (serial link, button on GPIO33, LED on GPIO26).
RUDRAN M (23BCE1269)	Device model, telemetry and Digital Twin	Capability definitions, templates, bindings, onboarding workflow and the site, building, space and device hierarchy. MQTT ingestion, validation, typed history and unit safety. Twin reported, desired and effective state, quality, readback and verification, domain events. Flyway migrations V1 to V39 and TimescaleDB retention.
SUDHAN SRIRAM (23BRS1264)	Operations Center, simulator, firmware and hardware	React 19 user interface (rule studio, dry-run page, Twin and pipeline views, analytics, capability and template administration, explanation panel) with the REST and STOMP client. Python simulator with three ESP32 profiles. Firmware, wiring, bill of materials and electrical safety plan.

A8.1 PUBLICATION AND PATENT STATUS

No research paper has been submitted or published, and no patent has been filed, for this work. No formal patent search was performed, and no claim of novelty or patentability is made. The contribution described in Chapter 4 is an engineering integration rather than a new algorithm. This appendix will be updated if a publication or patent is later obtained.

REFERENCES

1. Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M. (2015). Internet of Things: A survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials, 17*(4), 2347–2376.

2. Amazon Web Services. (n.d.). *AWS IoT Core developer guide: Device Shadow service*. https://docs.aws.amazon.com/iot/

3. Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). *Concrete problems in AI safety* (arXiv:1606.06565). arXiv.

4. Banks, A., & Gupta, R. (Eds.). (2014). *MQTT Version 3.1.1* (OASIS Standard). OASIS.

5. Berenson, H., Bernstein, P., Gray, J., Melton, J., O'Neil, E., & O'Neil, P. (1995). A critique of ANSI SQL isolation levels. *ACM SIGMOD Record, 24*(2), 1–10.

6. Eclipse Foundation. (n.d.). *Eclipse Ditto documentation*. https://eclipse.dev/ditto/

7. Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

8. Forgy, C. L. (1982). Rete: A fast algorithm for the many pattern/many object pattern match problem. *Artificial Intelligence, 19*(1), 17–37.

9. Fowler, M. (n.d.). *MonolithFirst*. https://martinfowler.com/bliki/MonolithFirst.html

10. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

11. Grieves, M., & Vickers, J. (2017). Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems. In F.-J. Kahlen, S. Flumerfelt, & A. Alves (Eds.), *Transdisciplinary perspectives on complex systems* (pp. 85–113). Springer.

12. Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2013). Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems, 29*(7), 1645–1660.

13. Hayes-Roth, F. (1985). Rule-based systems. *Communications of the ACM, 28*(9), 921–932.

14. Home Assistant. (n.d.). *Automation documentation*. https://www.home-assistant.io/docs/automation/

15. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys, 55*(12), Article 248.

16. Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.

17. Lee, E. A. (2008). Cyber physical systems: Design challenges. In *Proceedings of the 11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC)* (pp. 363–369). IEEE.

18. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474.

19. Light, R. A. (2017). Mosquitto: Server and client implementation of the MQTT protocol. *Journal of Open Source Software, 2*(13), 265.

20. Microsoft. (n.d.). *Azure Digital Twins documentation*. https://learn.microsoft.com/azure/digital-twins/

21. Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2nd ed.). Pragmatic Bookshelf.

22. OpenJS Foundation. (n.d.). *Node-RED documentation*. https://nodered.org/docs/

23. Ports, D. R. K., & Grittner, K. (2012). Serializable snapshot isolation in PostgreSQL. *Proceedings of the VLDB Endowment, 5*(12), 1850–1861.

24. Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.

25. Tao, F., Zhang, H., Liu, A., & Nee, A. Y. C. (2019). Digital twin in industry: State-of-the-art. *IEEE Transactions on Industrial Informatics, 15*(4), 2405–2415.

26. ThingsBoard. (n.d.). *Rule engine documentation*. https://thingsboard.io/docs/user-guide/rule-engine-2-0/overview/